

华南理工大学

South China University of Technology

第三讲：大模型推理

陈亦乐

计算机科学与工程学院

chenyile@scut.edu.cn

2026年9月16日

CHAPTER · PART 1

如何让模型效果更好

· 背景：为什么"直接作答"在复杂推理任务上失效

· 上下文学习（In-Context Learning）：给模型看几个例子

· 思维链（Chain-of-Thought）：先推理，后作答

· 自我一致（Self-Consistency）：多路径采样 + 多数投票

· 进阶增强：任务拆解 · 自我反思 · 思维树与 MCTS

"本节课中没有任何模型被训练——我们只用提示词，激发模型已有的能力。"

为什么"直接作答"不够?

复杂推理任务的核心挑战 · 大模型本质是 Next Token Prediction

"直接作答"模式

模型接收到问题后，直接输出答案，不展示思考过程。

"推理"模式

先给出最终答案前，生成一系列推理步骤，形成可追踪的思考过程。

核心挑战

① 复杂任务（数学、编程、规划）包含多层次逻辑结构与推理路径；

② 大模型本质是 Next Token Prediction，是"接龙"而非"解题"；

③ 直接作答需在极少 token 内一次性掌握所有推理链条，建模难度极高；

④ 引入显式中间步骤：问题 → 推理状态1 → 推理状态2 → … → 答案，**把建模难度摊平到多步**。



从一个失败案例看推理难点

问题

食堂买了 23 个苹果，用 20 个做了午餐，又买了 6 个，现在还剩多少个苹果?

直接作答

"还剩 27 个苹果。"

答案脱口而出、语气自信，但 23−20+6 = 9，**答错了**。

分步推理

"原有 23 个，用掉 20 个剩 3 个；又买 6 个，3+6 = 9 个。"

显式写出中间步骤，**答案正确**。

失败背后的四点思考

1. 模型作答高度依赖语言表层的统计模式，缺乏稳定的结构化推理路径；
2. 回答语法正确、表达通顺，逻辑层面却可能漏洞百出；
3. 根本原因在于缺乏对中间状态与逻辑转换的建模；
4. 问题结构稍作变化（加条件、引变量），模型的泛化能力便明显下降。

结论：应对复杂任务，必须显式建模思维过程——不能仅靠语言生成的统计规律。

上下文学习 ICL：给模型看几个例子

什么是 In-Context Learning

使用从类比中学习的思想：在输入中添加适当的示例文本，针对性地提高模型能力。

传统示范学习需要微调训练；ICL **不需要任何参数更新**，直接使用预训练好的语言模型，通过语义学习隐藏在示例中的模式，并据此做出预测。

One-Shot / Few-Shot

按示例数量区分：单样本提示（1 例）、少样本提示（K 例）；零样本则无示例。

ICL × CoT

示例中带上推理过程，就是 Few-Shot CoT——两者结合往往能取得更好效果。

完整提示词 = 任务描述 + K 个示例 + 新的 query

任务描述：请根据评论分析情感倾向：

评论：真是美味的食物！　情感：正面

评论：这东西太难吃了！　情感：负面

评论：这饭菜令人作呕！　情感：负面

评论：这顿饭真不错！　情感：？

↓

大语言模型（参数固定）

↓

正面

① 拼装输入

任务指令 + K 个"输入—输出"示例 + 新的问题，拼成一段提示词。

② 前向推理

参数完全冻结，只做一次普通的前向计算，没有任何梯度更新。

③ 直接预测

模型模仿示例的格式与分布，续写出新问题的答案。

本质：不训练模型，"现场教"模型任务的格式与分布

ICL 的起源：GPT-3 与 Few-Shot 范式

Language Models are Few-Shot Learners · Brown et al., NeurIPS 2020（arXiv:2005.14165）

GPT-3 参数量

175B

足够大的规模，让 ICL 能力"涌现"

论文的核心主张

· 不做任何微调，纯靠前向推理完成任务；

· 模型规模越大，上下文学习能力越强；

· Few-Shot 与微调模型的差距随规模迅速缩小。

Zero-Shot

只有任务描述，没有任何示例——完全依赖预训练知识。

One-Shot

只给 1 个示例，示范任务的输入—输出格式。

Few-Shot

给 K 个示例（通常几个到几十个），模型据此归纳任务模式——效果通常最好。

同一套权重，仅靠改变"给几个例子"，就能切换翻译、问答、算术、完形填空等截然不同的任务。

划时代意义：GPT-3 证明"不训练的示例学习"可行——提示词从此成为一种编程方式。

ICL 关键因素①：示例数量 × 模型规模

示例数量：边际收益递减

· 0 → 1 → 几例：性能**提升最显著**，格式对齐的作用立竿见影；

· 继续增加示例：收益迅速衰减，很快触及天花板；

· 示例占用宝贵的上下文窗口——不是越多越好，而在"精"。

工程经验：从 4~8 个精心挑选的示例起步，往往比随手堆 30 个更有效。

模型规模：ICL 是"涌现"能力

· 小模型几乎**不具备** ICL 能力——给了示例也不模仿；

· 跨过规模阈值后，ICL 效果陡然出现并持续增强（与 CoT 类似的涌现现象）；

· 经过指令微调（Instruction Tuning）的模型，ICL 表现更进一步。

启示：同样的提示词，换更大的模型可能就是最大的"提示优化"。

两个前提：有几个好例子 + 一个够大的模型——ICL 才真正成立。

ICL 关键因素②：选择 · 顺序 · 格式

示例选择

与当前问题语义越相似的示例，帮助越大；可用嵌入检索从候选池中动态挑选最相关的 K 个。

示例顺序

模型对排列敏感，存在"近因效应"——关键示例放后面常被更多利用；重要任务可多试几种排列。

格式一致

所有示例共用同一套"输入—输出"模板，标签写法、分隔符统一，模型才能稳定模仿。

标签分布

各类标签均衡覆盖、标签空间完整展示，避免模型被示例中的类别偏置带偏。

常见误区

随手从训练集抓 K 条 → 示例与问题无关、格式不一、类别失衡，ICL 收益大打折扣。

正确姿势

像策展数据集一样策展示例：相关、一致、均衡——必要时为每个 query 动态检索。

示例的质量与组织方式，往往比示例的数量更决定 ICL 的上限。

模型真的"看懂"示例了吗①：标签打乱实验

Rethinking the Role of Demonstrations（arXiv:2202.12837）· 2022 年的认知

实验设计

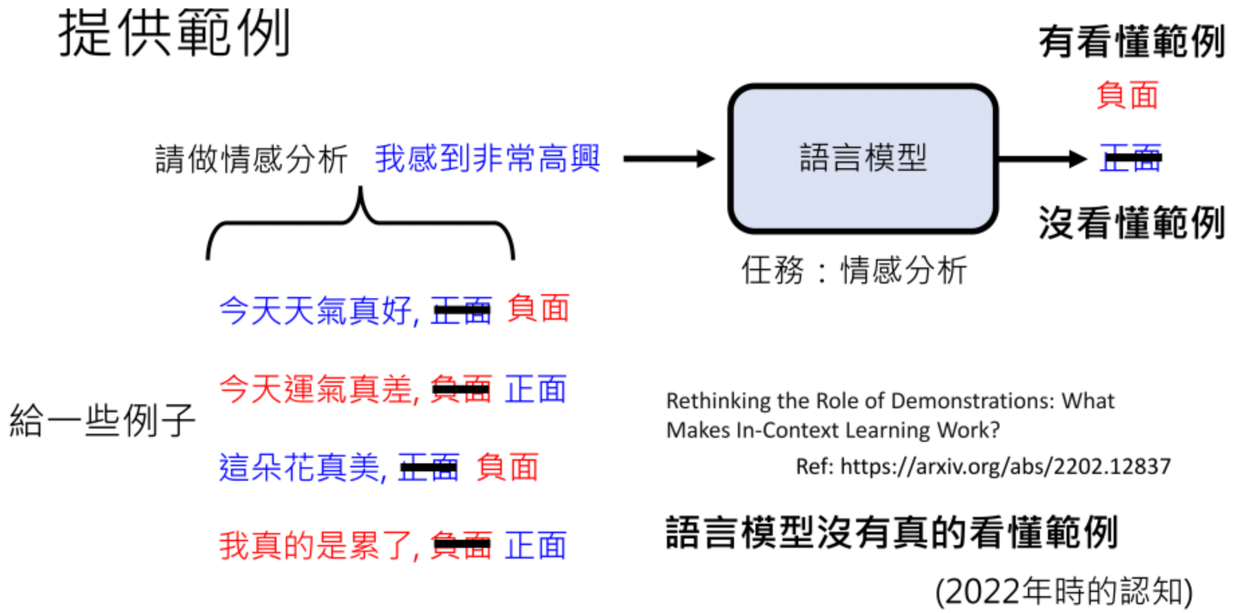
把示例中的标签随机替换（"正面"改成"负面"），再让模型做同样的任务。

出人意料的结果

性能几乎不下降——模型并不依赖示例中真实的"输入→标签"映射。

当时的结论

示例真正提供的是：标签空间、输出格式、输入分布——模型没有真的"看懂"示例。



Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?

Ref: https://arxiv.org/abs/2202.12837

語言模型沒有真的看懂範例

(2022年時的認知)

来源：Hung-yi Lee, Machine Leaning Course

ICL 在 2022 年的画像：示例是"格式说明书"，而不是"知识教材"。

模型真的"看懂"示例了吗②：大模型不一样

Larger language models do in-context learning differently（arXiv:2303.03846）

升级版实验

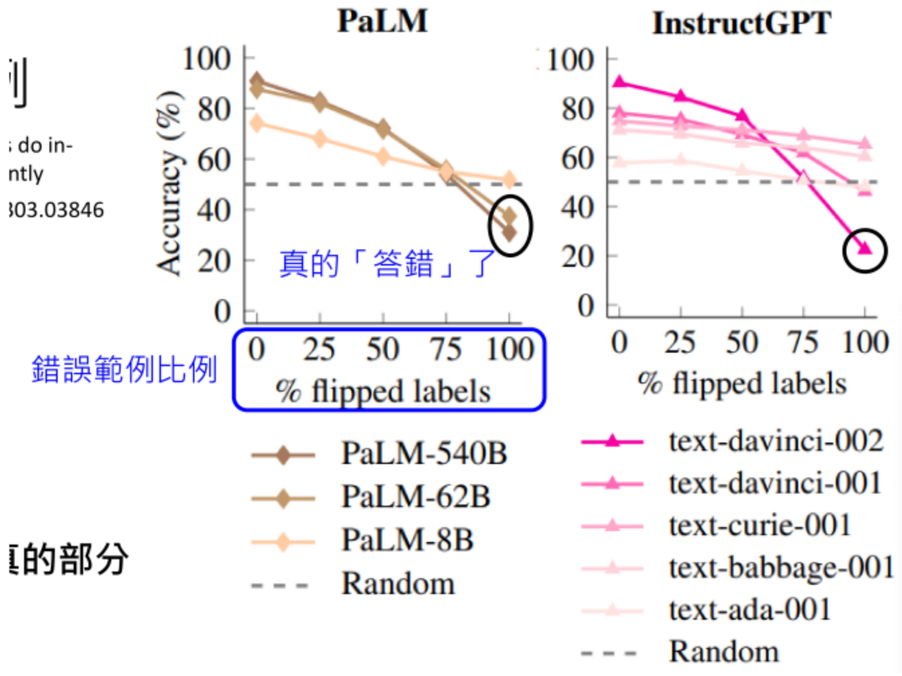
不再全打乱，而是逐渐提高错误标签的比例（0% → 100%），观察不同规模模型的准确率曲线。

关键发现

错误示例超过阈值后，大模型（PaLM-540B、text-davinci-002）的准确率**跌破随机基线**——真的被"带偏"、真的答错了；小模型则无动于衷。

结论更新

最强的模型确实在部分利用示例中的真实映射——"会跑偏"恰恰说明它"在读"。



来源：Hung-yi Lee, Machine Leaning Course

规模改变性质：小模型看格式，大模型连示例的"内容"也在学。

长上下文 ICL：把"整本教材"放进提示词

Gemini 1.5 In-context Learning · 上下文窗口 = 新的教材容量

实验设定

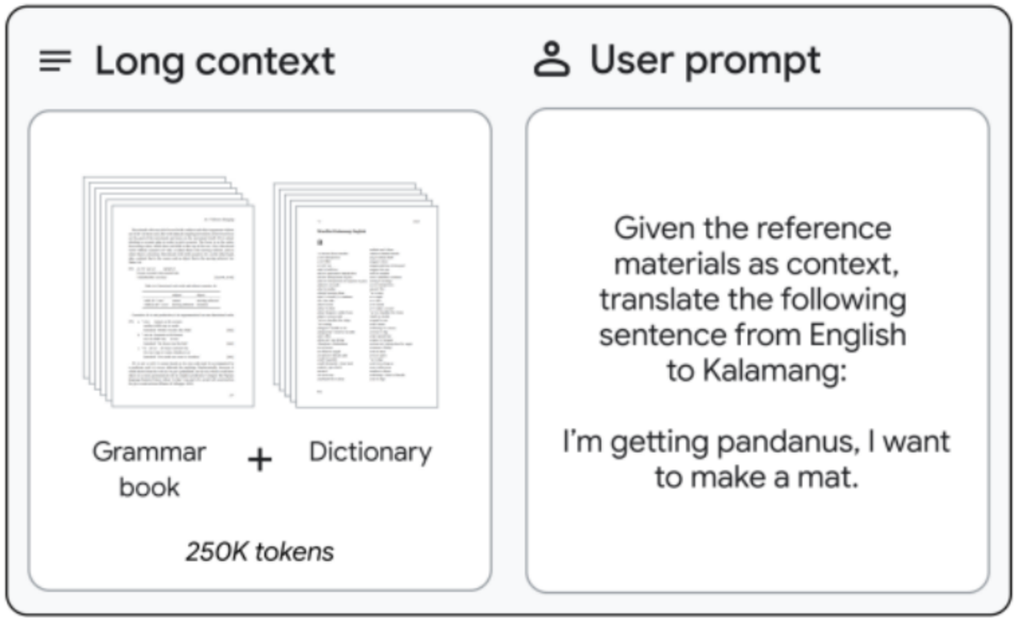
给 Gemini 1.5 输入一本 25 万 token 的语法书 + 词典，目标语言是互联网上极少语料的罕用语言 Kalamang。

结果

模型在不做任何训练的情况下，学会把英语翻译成 Kalamang，翻译质量接近人类学习者水平（人工评分，满分 6 分）。

启示

当上下文窗口足够长，"给几个例子"就升级为"给一整本教材"——知识可以通过提示词临时装载。



ICL 的边界正在被上下文长度重新定义：示例 → 范例集 → 整本教材。

思维链 CoT：先推理，后作答

Chain-of-Thought Prompting · Wei et al., NeurIPS 2022

核心思想

让模型显式写出多个中间推理步骤，逐步构建通向最终答案的逻辑路径——输出更透明、逻辑更稳健，也便于后续检查和改进。

Few-Shot CoT

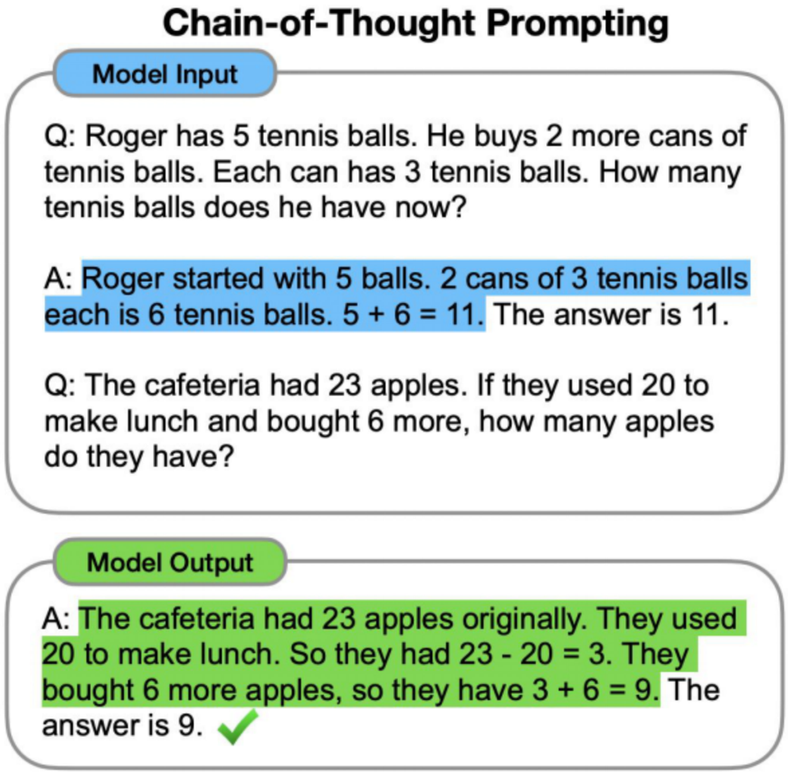
在 ICL 示例中手写推理过程，模型模仿示例的"解题格式"作答。

Zero-Shot CoT

不给示例，只在问题后加一句链式思考提示词，即可激发分步推理。

最常见的"魔法咒语"

"Let's think step by step." / "让我们一步一步思考。"

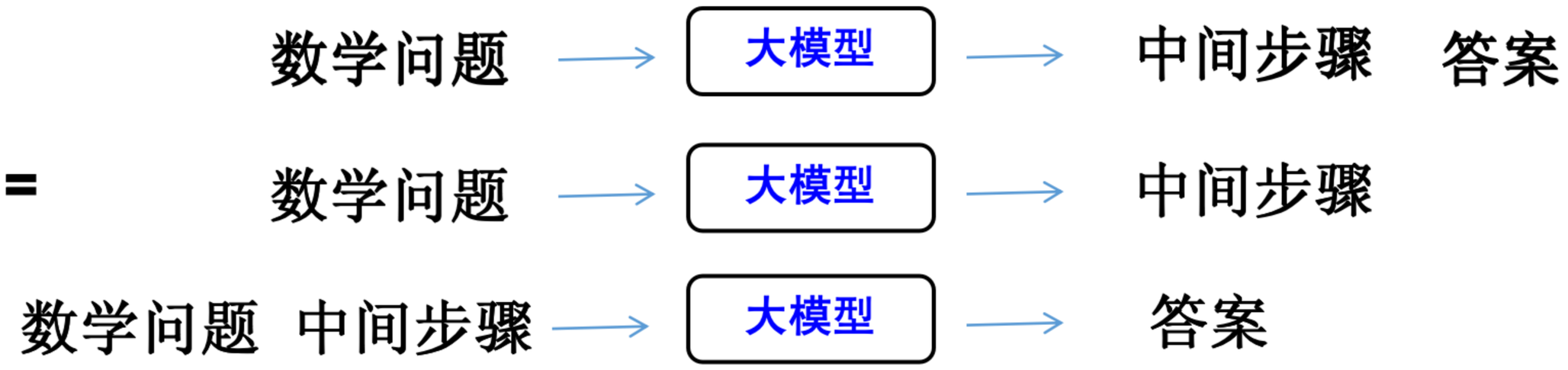


Few-Shot CoT 示例：答案前先写推理过程（Wei et al., 2022）

本质：把"一步到位"的硬骨头，拆成一串小步骤——每步都在模型的能力圈内。

拆解来看：为什么"先想后答"会有用

一次难调用 = 两次易调用



整体视角

"问题 → 步骤 → 答案" 等价于把一次难调用，拆成两次各自更简单的调用。

第一步：步骤

把自然语言问题翻译成算式 / 推理步骤——只负责"想"，不负责"答"。

第二步：求解

基于已列出的式子算出答案——上下文里已有中间结果，难度骤降。

注：新一代模型解数学题默认就会"列式"——所以 CoT 提示对它们的额外帮助变小，对老模型收益更大。

零样本 CoT："让我们一步一步思考"

Large Language Models are Zero-Shot Reasoners · Kojima et al., 2023

No.	Category	Template	Accuracy
1	instructive	Let's think step by step.	**78.7**
2		First, (*1)	77.3
3		Let's think about this logically.	74.5
4		Let's solve this problem by splitting it into steps. (*2)	72.2
5		Let's be realistic and think step by step.	70.8
6		Let's think like a detective step by step.	70.3
7		Let's think	57.5
8		Before we dive into the answer,	55.7
9		The answer is after the proof.	45.7
10	misleading	Don't think. Just feel.	18.8
11		Let's think step by step but reach an incorrect answer.	18.7
12		Let's count the number of "a" in the question.	16.7
13		By using the fact that the earth is round,	9.3
14	irrelevant	By the way, I found a good restaurant nearby.	17.5
15		Abrakadabra!	15.5
16		It's a beautiful day.	13.1
-		(Zero-shot)	17.7

不同提示词模板的准确率对比（MultiArith，InstructGPT）

MultiArith 准确率

17.7% → **78.7%**

零样本 → "Let's think step by step"

咒语观察

· "Let's think step by step." 78.7 — 最强零样本咒语

· "Let's think about this logically." 74.5

· "Don't think. Just feel." 18.8 — 抑制思考几乎归零增益

新模型越来越强：老模型需要咒语引导，新模型默认就会分步推理——模型理应随时使出全力。

一句话、零成本：在问题后加 "Let's think step by step"，数学推理准确率翻四倍。

零样本 CoT 的实现：两阶段提示

Large Language Models are Zero-Shot Reasoners Kojima et al., 2023 · 先"提取推理"，再"提取答案"

1 推理提取：问题 + "Let's think step by step"

System: 你是一个数学智能体，你需要逐步推理，然后给出答案。

Q: 若 $f(x) = \frac{3x-2}{x-2}$ ，那么 $f(-2) + f(-1) + f(0)$ 的值是多少？请将你的答案化为最简分数

2 答案提取：拼接推理 + "Therefore, the answer is"

要分别计算每个函数值，然后将结果相加。

首先，我们计算 f(-2)：

$f(-2) = \frac{3(-2)-2}{-2-2} = \frac{-6-2}{-4} = \frac{-8}{-4} = 2$

接下来，我们计算 f(-1)：

$$f(-1) = \frac{3(-1)-2}{-1-2} = \frac{-3-2}{-3} = \frac{-5}{-3} = \frac{5}{3}$$

然后，我们计算 f(0)：

$$f(0) = \frac{3(0)-2}{0-2} = \frac{-2}{-2} = 1$$

现在，我们将这些值相加：

$$f(-2) + f(-1) + f(0) = 2 + \frac{5}{3} + 1$$

为了将这些数相加，我们需要一个公共分母。2 和 1 可以写成 $\frac{6}{3}$ 和 $\frac{3}{3}$，分别。因此，我们有：

$$2 + \frac{5}{3} + 1 = \frac{6}{3} + \frac{5}{3} + \frac{3}{3} = \frac{6+5+3}{3} = \frac{14}{3}$$

为什么要拆两步：第一阶段让模型自由生成长推理，第二阶段用固定触发语把答案"收束"成规范格式，便于自动评测与投票。

工程要点：答案提取提示词要与任务输出空间匹配（阿拉伯数字 / 选项字母 / 是否），否则会"想对答错"。

一句咒语背后，是"先生成、后收束"的两段式设计——这也是后续 Self-Consistency 的标准接口。

CoT 变体：让示例与链条更聪明

从"手写示例"到"自动构造" · Manual-CoT → Auto-CoT

Manual-CoT

人工为每个任务手写带推理链的示例。效果稳定，但编写成本高、难以规模化。

基线做法

Auto-CoT

① 用聚类把问题库分成若干类；② 每类选代表问题；③ 用 Zero-Shot CoT 自动生成推理链作为示例——自动构造"教材"。

Zhang et al. 2023

Complexity-CoT

优先选择"推理步骤最多"的示例与输出：越复杂的链条，越能激发模型的完整推理。

Fu et al. 2022

维度	Zero-Shot CoT	Manual Few-Shot CoT	Auto-CoT
示例来源	无示例	人工编写	自动生成
成本	极低	高（逐题手写）	低（一次构造，多次复用）
典型效果	良好	最好但难扩展	接近甚至追平 Manual

趋势：示例构造也在"自动化"——用模型的 CoT 能力，为 CoT 自己造教材。

CoT 为什么有效①：推理路径本就存在

CoT-Decoding · Wang & Zhou, 2024 / CoT 是规模的涌现能力

[GSM8K] *Kylar went to the store to buy glasses for his new apartment. One glass costs $5, but every second glass costs only 60% of the price. Kylar wants to buy 16 glasses. How much does he need to pay for them?*
Greedy path:
$k = 0$: \$**60.00** (0.029)
Alternative top-k paths:
$k = 1$: **60** (0.058)
$k = 2$: Kylar needs to pay \$**60** for 16 glasses. (0.058)
. . .
$k = 7$: If Kylar buys 16 glasses, he will pay \$**60**. (0.032)
$k = 9$: We can calculate the price of 16 glasses by multiplying the price of one glass by 16. However, we need to remember that every second glass costs only 60% of the price. Therefore, we need to multiply the price of one glass by 16 and then subtract 40% of the price of 8 glasses. 16 x 5 = 80 8 x 5 = 40 40 x 0.4 = 16 80 – 16 = 64 Kylar needs to pay \$**64** for 16 glasses. (0.994)

GSM8K 例题：greedy 直接答错；替代 top-k 路径中自发出现 CoT，答案置信度大幅提升

CoT-Decoding 实验

只对首个 token 按 top-k 分支，后续照常 greedy 解码——不做任何 CoT 提示，替代路径中**自发出现思维链**。

关键启示

推理路径天然存在于预训练模型中，只是被标准 greedy 解码掩盖——提示词和解码策略都能把它"挖"出来。

涌现能力

CoT 收益在 100B+ 模型上才显著；小模型反而产生不合逻辑的链条，不升反降。

本质：推理能力一直藏在模型里——greedy 解码只是没有找到它。

CoT 为什么有效②：结构重于正确性

Towards Understanding CoT (Wang et al., 2022) ·

实验发现：逻辑错误也有效

即使示例中**故意含逻辑错误**，模型性能仍大幅提升到 **80–90%**——CoT 的关键在于结构，而非示例的逻辑正确性。

任务分解，降低负担

复杂问题拆成小步骤，减少跳步出错，提高解题稳定性。

路径显性，便于分析

每步推理可见，提升可解释性，为"反思与调试"奠定基础。

本质：CoT 不是灌输推理知识，而是启发模型按步骤表达已有能力——关键要素是内容相关性 + 步骤顺序清晰。

CoT 为什么有效③：经验的局部性

Why Think Step by Step · Prystawski et al., 2023

训练数据是"局部"的

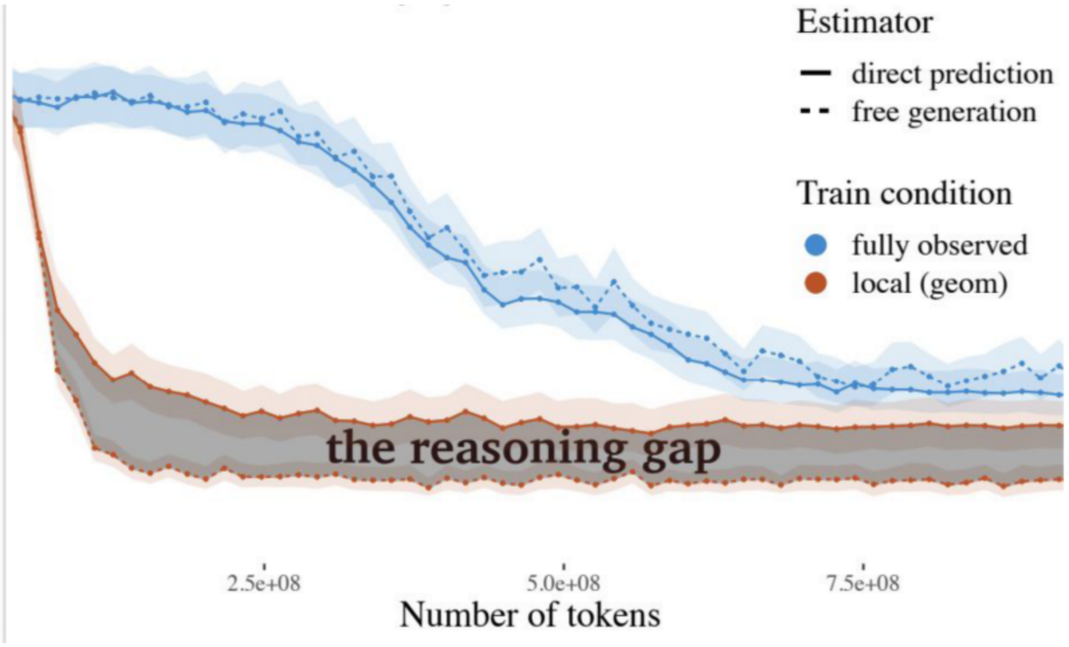
语料中的知识以重叠的局部变量簇形式出现：变量两两经常同框，但任意两个变量未必同时出现过。

链条把局部串成全局

逐步推理沿着中间变量逐跳传播，才能对"从未同框"的变量对得出结论——一步到位反而做不到。

理论贡献

在贝叶斯网络框架下数学上证明了 reasoning gap 的存在：只要知识是局部的，链式推理就严格优于直接估计。



局部经验模型：观测变量经中间变量逐跳连接到目标变量

冷静看待：CoT 并非万能

① "想多了"反而变差

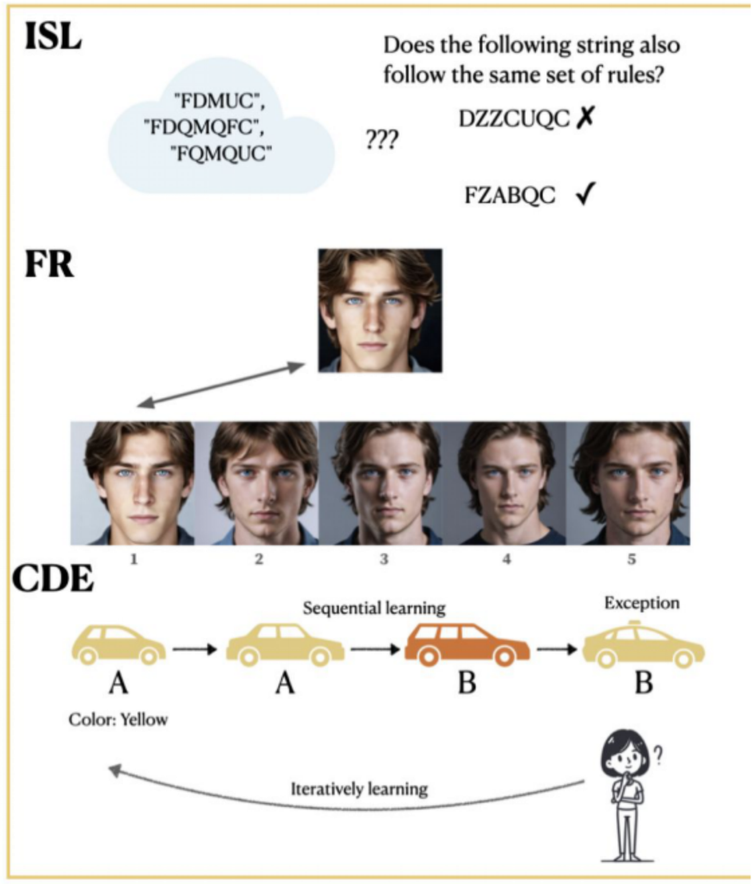
Mind Your Step（Liu et al., 2025）：在人类也会"越想越错"的任务上（如依赖直觉的隐性统计、模式分类），CoT 会显著降低模型性能。

启示：CoT 适合"步骤型"推理，不适合"直觉型"判断。

② 推理链不一定"说真话"

Reasoning Models Don't Always Say What They Think（Chen et al., Anthropic, 2025）：答案实际被 hint 影响时，CoT 中披露 hint 的比例不足 20%——模型系统性地"用了但不承认"。

启示：CoT 提供了可解释性的"窗口"，但链条 ≠ 真实决策原因。



部分任务上 CoT 反而拉低性能（Liu et al., 2025）

使用前先问两个问题：这个任务适合分步思考吗？我能相信这条链吗？

基础链式思维的三个局限

线性单链的固有缺陷 · 每个短板对应一类增强策略

① 缺乏全局规划能力

前向线性结构，缺失反向调整能力，缺乏全局协调——任务稍复杂就容易遗漏中间目标。

→ 规划策略：Plan-and-Solve / Least-to-Most

② 易受错误积累影响

每一步依赖上一步，早期出错则后续全程建立在错误基础上；推理链越长，越难维持逻辑一致，准确率迅速下降。

→ 反思策略：Reflexion 自我纠错

③ 难以覆盖复杂推理路径

结构单一，不支持并行、回溯、条件分支等非线性逻辑；无法动态选择路径，也不能在方案间跳转。

→ 搜索策略：ToT / MCTS（LATS）

前瞻 + 后顾

规划：推理之初设定全局目标，中途回看动态调整

迭代纠错

反思：评估-反馈-记忆，把错误变成下次的经验

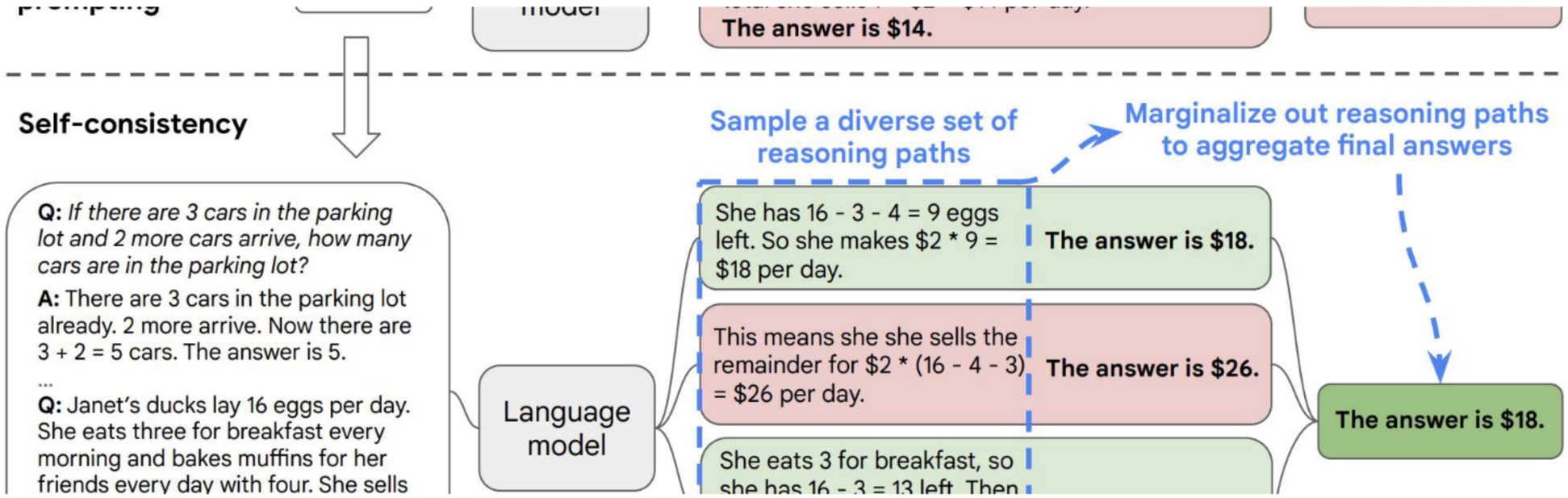
多路探索

搜索：同时探索多路径，评估筛选，失败可回溯

单链思维是推理增强的起点，不是终点——规划、反思、搜索，各补一个短板。

自我一致 Self-Consistency：多路径采样 + 投票

Wang et al., ICLR 2023 · 用集成思想替代贪婪解码



greedy 只走一条路；采样多条多样化推理路径，多数投票聚合最终答案（Wang et al., 2022）

① 多路径生成

用 CoT 提示 + 采样式解码（temperature / top-k），对同一问题生成 N 条不同推理链。

② 投票决策

从每条链中提取答案，多数投票，选出现频率最高的结果作为最终输出。

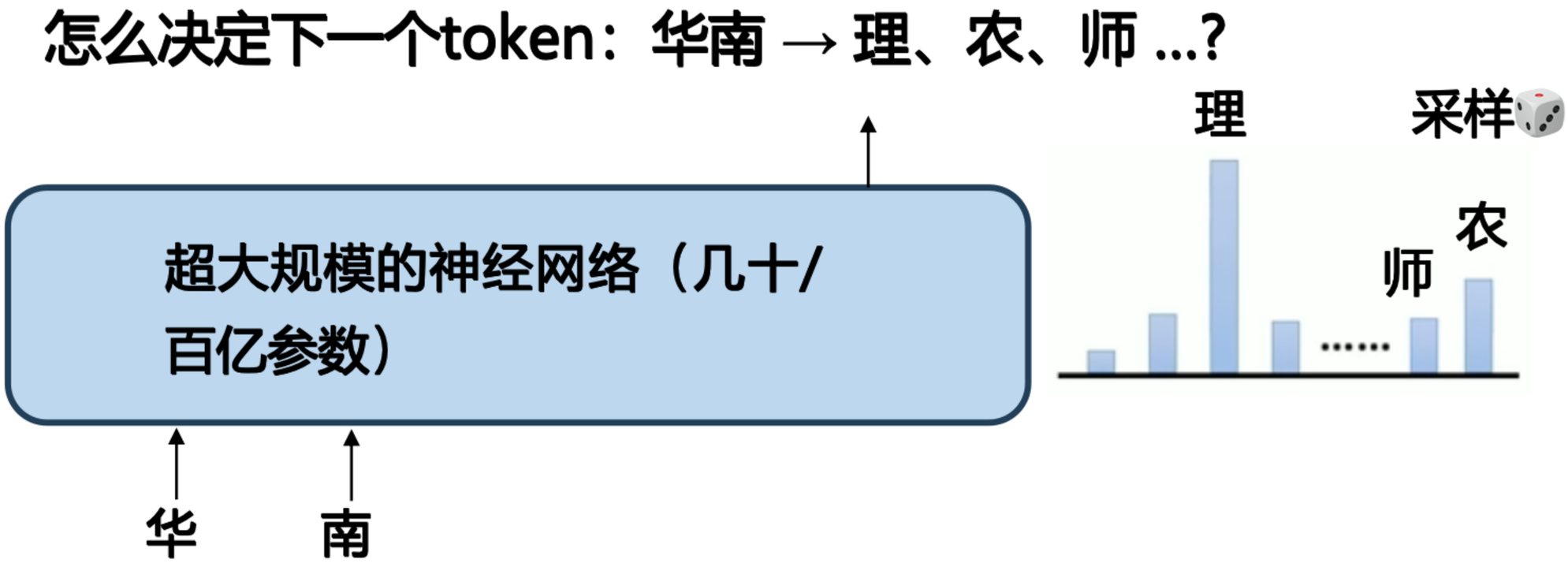
③ 动机：替换 greedy

单链 greedy 任一步出错即全盘皆输；采样让多条路径互相"纠错"。

本质：同一个问题，答案可以掷骰子多次——多数人的选择，往往是对的。

前奏：为什么同一问题，每次答案都不同？

采样解码的随机性 · 随机性既是麻烦，也是资源



每个位置都从概率分布中采样下一个 token——每次生成的轨迹都可能不同

随机从哪来

温度采样 / top-k / top-p：模型输出的是概率分布，解码是一次"掷骰子"。

麻烦的一面

结果不可复现、单次答案不可靠——推理任务里一次出错就是**满盘皆输**。

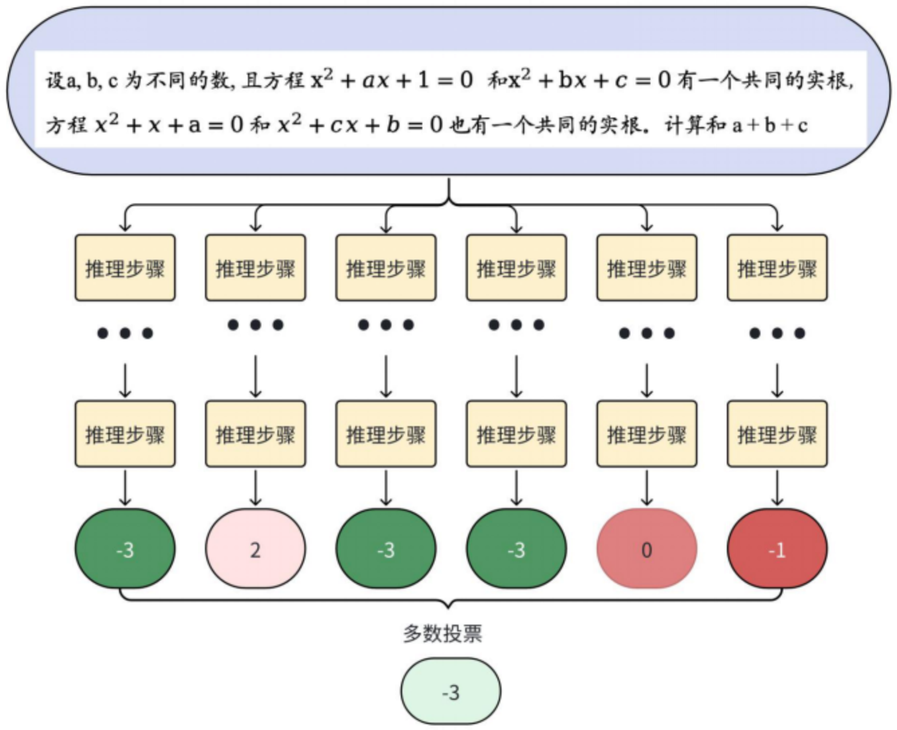
资源的一面

多次采样 = 多条多样化推理路径——正好为"投票"提供原材料。

思路转换：与其追求"一次答对"，不如把随机性收集起来——让多数路径替我们把关。

Self-Consistency：为什么有效，效果几何

错误分散 · 正确收敛 · 用算力换准确率



二次方程实例：多条推理路径 → 多数投票得 -3

+17.9%
GSM8K

+11.0%
SVAMP

+12.2%
AQuA

增益原理

· 错误推理路径各异，错误答案分散在多个选项中，难以形成多数；

· 正确推理形式不同，但多数收敛到同一个答案——具备"频率优势"。

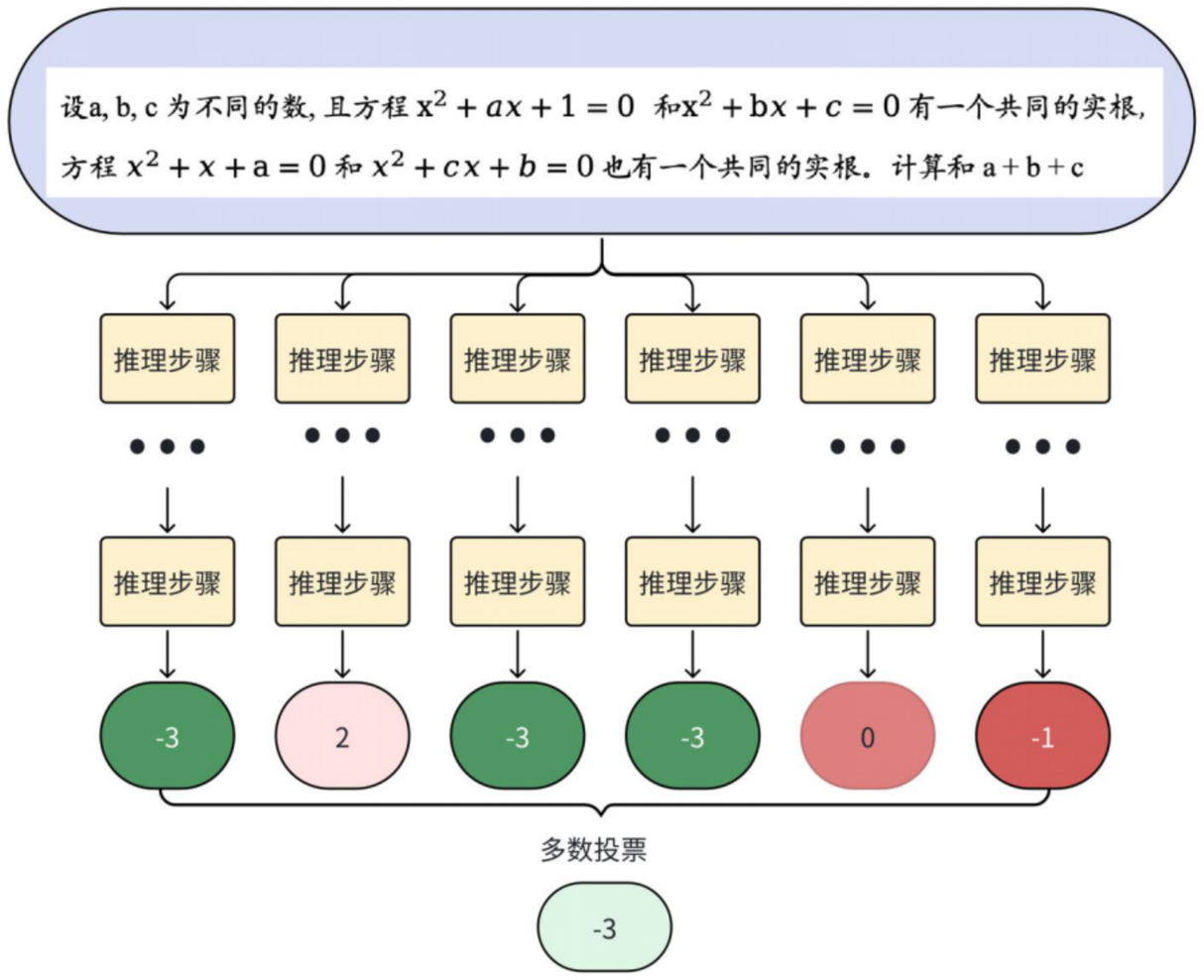
N 越大，提升越大 → Test-Time Scaling

采样数 N 增加可持续提升性能；代价是 N 倍推理算力。无需修改模型结构，即可提升推理稳定性与准确率。

本质：用 N 倍推理算力，换显著更稳的准确率——test-time scaling 的经典范式。

Self-Consistency 实践要点与成本权衡

完整流程 · 采样数 · 温度 · 答案提取



完整流程：同一问题采样多条推理链 → 提取各链答案 → 多数投票输出

采样数 N

常用 5~40；收益随 N 增加但递减，N=10 左右性价比最高。

温度设置

0.7~1.0：太低路径趋同失去多样性，太高链条质量下降。

答案提取

用 "The answer is" 等触发语收束格式，保证投票口径一致。

成本权衡

N 倍推理算力换准确率——适合有唯一答案、可自动提取答案的任务。

一句话工程版：CoT 提示 + 温度 0.7 采样 10 次 + 规则提取答案 + 众数投票。

基础链式思维的三个局限

线性单链的固有缺陷 · 每个短板对应一类增强策略

① 缺乏全局规划能力

前向线性结构，缺失反向调整能力，缺乏全局协调——任务稍复杂就容易遗漏中间目标。

→ 规划策略：Plan-and-Solve / Least-to-Most

② 易受错误积累影响

每一步依赖上一步，早期出错则后续全程建立在错误基础上；推理链越长，越难维持逻辑一致，准确率迅速下降。

→ 反思策略：Reflexion 自我纠错

③ 难以覆盖复杂推理路径

结构单一，不支持并行、回溯、条件分支等非线性逻辑；无法动态选择路径，也不能在方案间跳转。

→ 搜索策略：ToT / MCTS（LATS）

前瞻 + 后顾

规划：推理之初设定全局目标，中途回看动态调整

迭代纠错

反思：评估-反馈-记忆，把错误变成下次的经验

多路探索

搜索：同时探索多路径，评估筛选，失败可回溯

单链思维是推理增强的起点，不是终点——规划、反思、搜索，各补一个短板。

Plan-and-Solve：先制定计划，再分步执行

Wang et al., ACL 2023 · 零样本规划提示

(a) 规划阶段

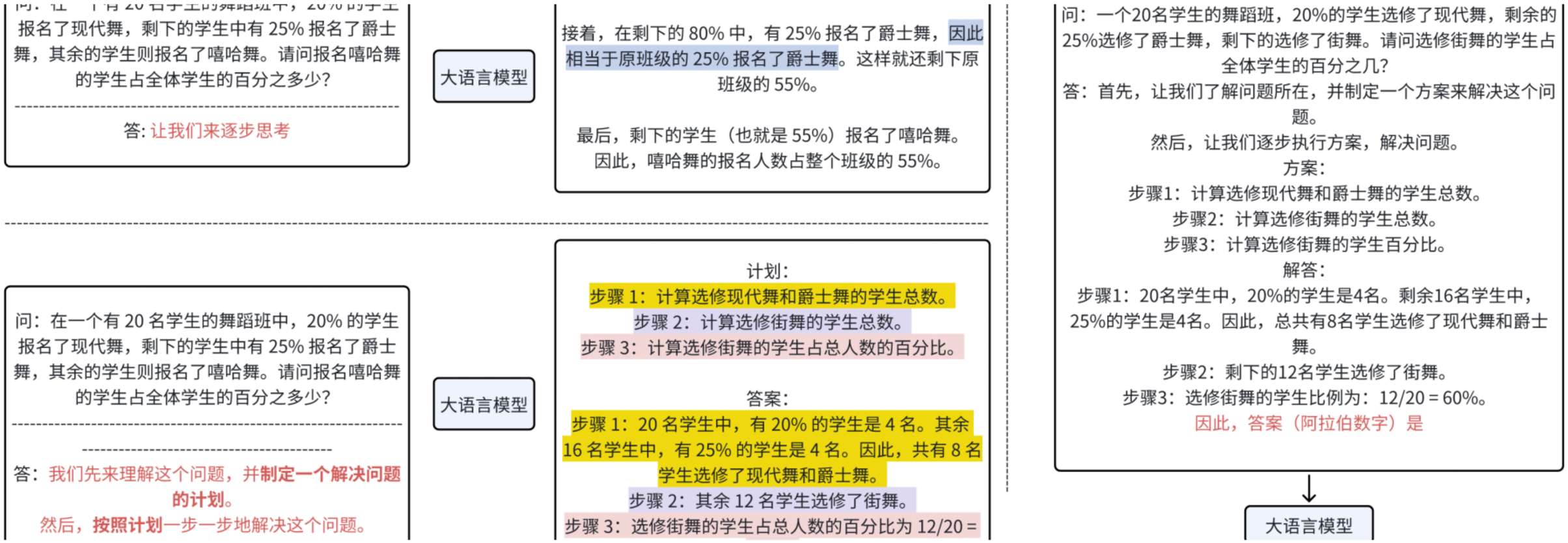
先让模型生成结构化计划："让我们首先通过将问题分解为子步骤来制定计划。"

(b) 执行阶段

把计划的每一步作为单独输入，依次输出思维链与中间结果，最终汇总答案。

PS+ 增强版

追加"提取相关变量""注意中间计算"等细粒度指令，零样本即可超越 Zero-shot-CoT。



Plan-and-Solve 两阶段示意：先规划出子步骤，再逐步求解（Wang et al., 2023）

本质：用"元推理"先想清楚怎么解，再动手解——补上链式思维缺失的全局规划。

反思机制：让模型检查自己的错误

Reflexion (Shinn et al., NeurIPS 2023) · 得答案难，验证答案易

为什么反思有效

很多问题"得到答案难，**验证答案是否正确容易**"——鸡兔同笼 35 头 94 脚，答案对不对一算便知。就像写完考卷再检查一遍，总能查出错误。

1 行动者 Actor

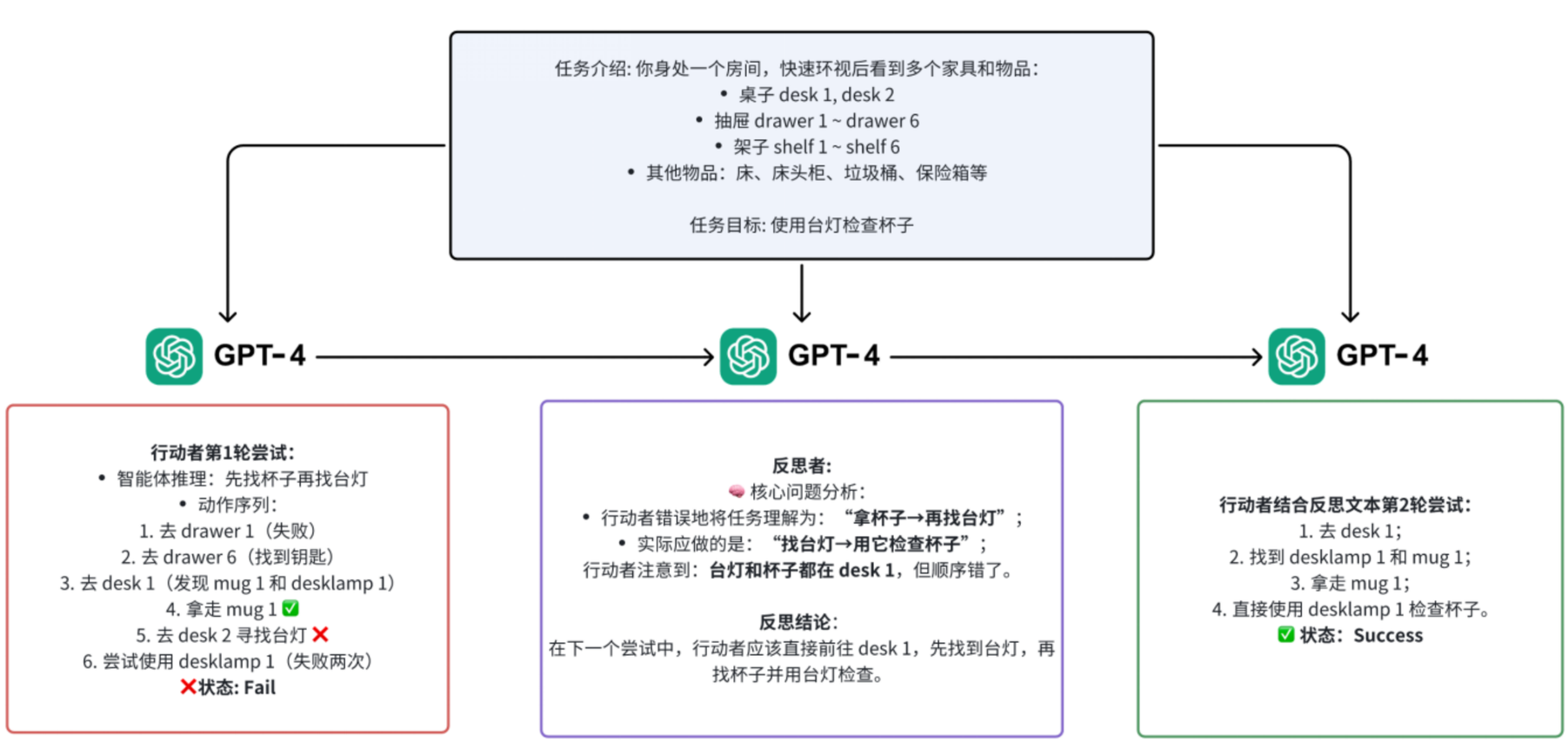
以 CoT / ReAct 模式与环境交互，生成行为轨迹。

2 评估者 Evaluator

对轨迹打分（是否成功、通过多少测试用例）。

3 反思者 Self-Reflection

生成文字版"反思总结"写入记忆池，指导下一轮执行。



本质：语言的强化学习——不改模型参数，只把"失败的经验"写进记忆。

思维树 ToT：把推理组织成一棵树

Tree of Thoughts · Yao et al., NeurIPS 2023

① 思维分解

把解题过程拆成若干"思维步"，每步是一个候选节点。

② 思维生成

每个节点多次采样，产生多个候选思维分支。

③ 思维评估

让 LLM 自评每个分支"通向解的潜力"（sure / maybe / impossible）。

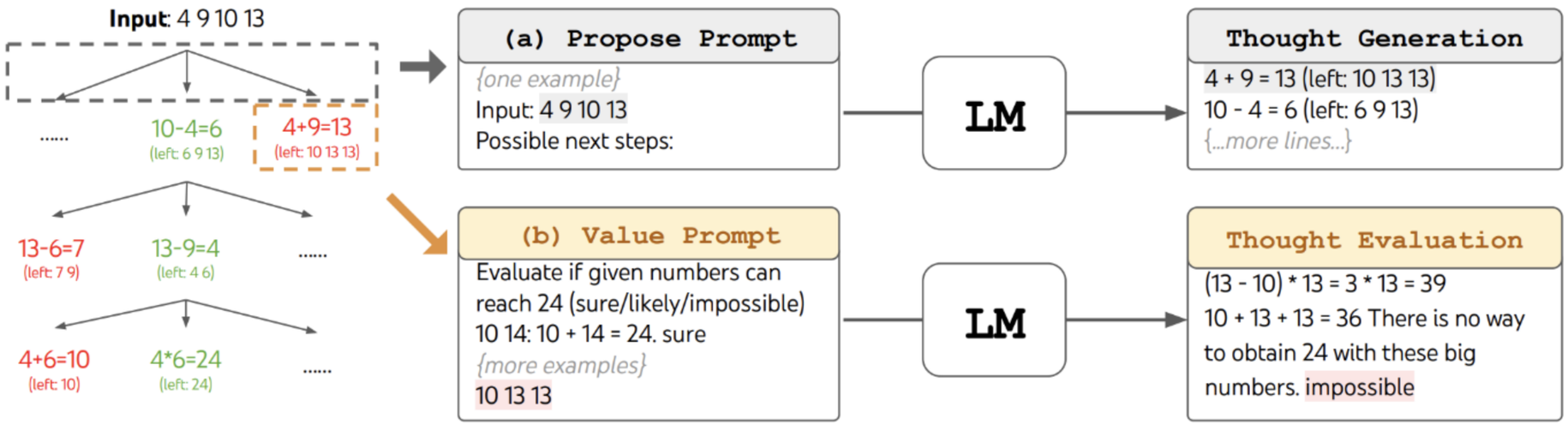
④ 搜索回溯

BFS / DFS / Beam 系统探索；分支无前景则剪枝回退。

24 点游戏战绩（GPT-4）

4% → **74%**

CoT 提示仅 4% 成功率；ToT 通过多路探索 + 剪枝提升到 74%。



24 点游戏：Propose → Value → 剪枝搜索

本质：从"一条链走到黑"升级为"探索一片森林"——可并行、可评估、可回溯。

LATS：语言模型 × 蒙特卡洛树搜索

Language Agent Tree Search · Zhou et al., 2023

MCTS 四阶段

选择 → 扩展 → 模拟 → 回传：兼顾随机探索与系统评估，在巨大决策空间中高效分配算力。

LLM 的双重角色

语言模型既生成候选动作（子问题解法、下一步操作），又充当价值评估器为路径打分——搜索框架提供结构，LLM 提供智能。

相比 CoT / ToT

支持"思考—尝试—修正"闭环，可与外部环境交互、利用反馈更新策略——更强的交互性与策略学习能力，适合反馈不确定的复杂任务。

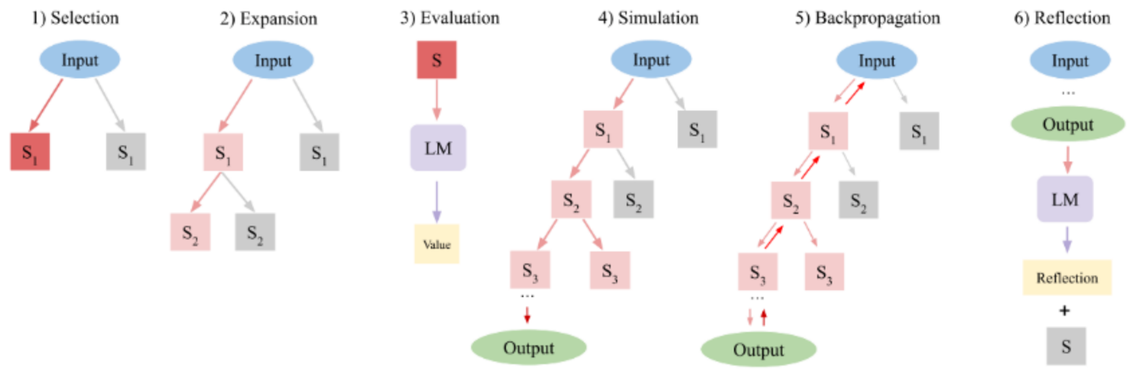


Figure 2. Overview of the six operations in LATS. A node is *selected*, *expanded*, *evaluated*, then *simulated* until a terminal node is reached, and then the resulting value is *backpropagated*. If the trajectory fails, a *reflection* is generated and used as additional context for future trials. These operations are performed in succession until the budget is reached or the task is successful.

LATS 六种操作：选择 / 扩展 / 评估 / 模拟 / 回传 / 反思

典型应用：Web 导航（WebShop）、工具调用、代码生成等需要多轮试错的智能体任务。

本质：经典搜索算法提供"骨架"，语言模型提供"血肉"——推理、行动、规划在此统一。

方法全景对照与选型建议

六类"不训练模型"的推理增强方法 · 核心思想 · 代价与适用场景

方法	核心思想	代表工作	适用场景	代价
ICL 上下文学习	示例示范格式与分布	GPT-3 (2020)	格式固定的分类 / 抽取	低（占上下文）
CoT 思维链	显式中间推理步骤	Wei et al. (2022)	数学 / 逻辑多步推理	低
Self-Consistency	多路径采样 + 多数投票	Wang et al. (2023)	有唯一答案的推理题	N 倍算力
任务拆解 / 规划	先计划后执行、由易到难	PS / L2M (2023)	多子任务、比示例更难	中（多轮调用）
反思 Reflexion	评估—反思—记忆迭代	Shinn et al. (2023)	可自动验证的智能体任务	中（多轮开销）
搜索 ToT / LATS	树搜索探索 + 评估回溯	Yao / Zhou (2023)	规划类、探索空间大	高（×10 以上）

先 CoT，再投票

零成本起步，唯一答案任务加 Self-Consistency

不够就拆

任务太复杂 → 拆解 + 规划，化整为零

错了再反思

有验证器 → Reflexion 迭代纠错

仍不行，上搜索

探索空间大 → ToT / MCTS，用算力换质量

选型口诀：先 CoT，再投票；不够就拆，错了再反思，仍不行就上搜索。

CHAPTER · PART 2

加快语言模型生成速度

· 自回归生成与 Prefill / Decode 两阶段瓶颈

· 解码策略：Greedy · Beam · Top-k · Top-p · Temperature

· KV-Cache：架构 / 内存 / 注意力 / 硬件 四层优化

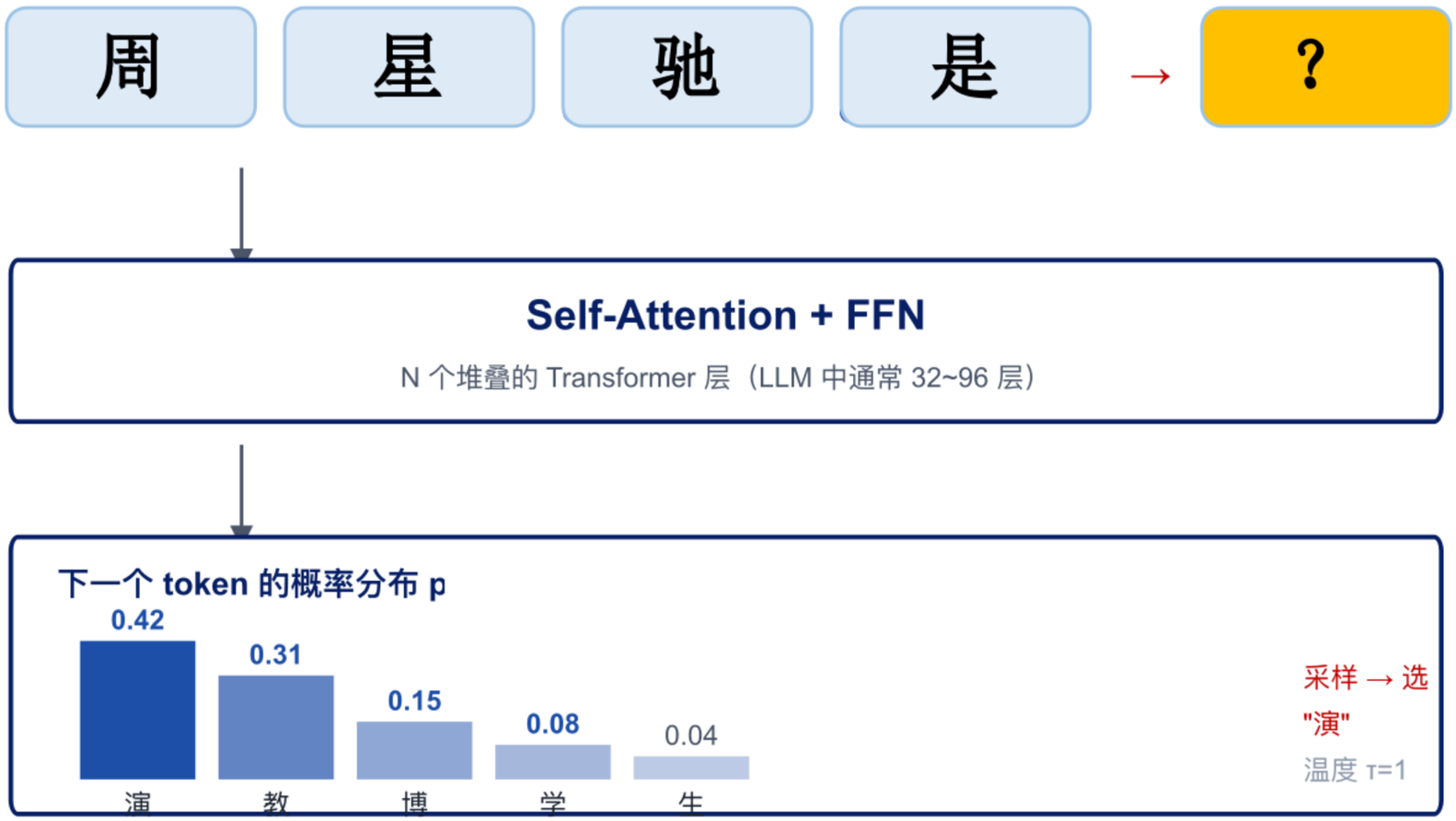
· Speculative Decoding：小模型预判 + 大模型验证

· 系统视角：Prompt Caching · 推理引擎 · 未来方向

"再强大的模型，如果响应慢到无法交互，就无法落地。"

自回归生成：大模型如何"接龙"

Autoregressive Generation · 文字接龙的工程实现



核心机制

给定上文 $x_1 \ldots x_n$，模型输出分布 $p(x_{n+1})$，采样得到下一个 token。接到上文末尾，循环直到 [END] 或达到最大长度。

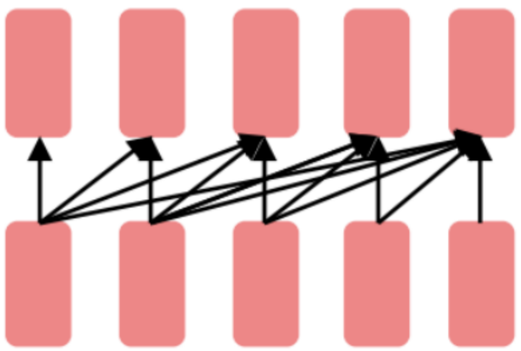
① 下一个 token 的概率基于整个上文，所以需要 Self-Attention。
② 每生成一个新 token 都要重新"看一遍"历史——这是 KV-Cache 要解决的冗余。
③ 采样策略的选择，直接影响生成速度与文本质量。

本质：生成一个 token 的时间 ≈ 单次 forward pass 时间 × token 数。每个 token 都独立可被优化。

模型预训练

■ 基于Transformer的预训练类型

- Decoder（GPT系列，现在几乎所有的大模型）



- 语言模型的定义
- 对语言生成最适配，不能看到未来的内容（对应Transformer模型Decoder中的masked self-attention部份）

来源：Stanford CS224N

生成式 AI 的统一视角

文字 · 图像 · 语音 — 三种模态的共同本质

文字

基本单位：Token

词表大小：32K (Llama 2)
单次生成 token 数：100~4K
接龙次数：数百次

工程优化最成熟、最容易落地的模态

图像

基本单位：Pixel

颜色深度：8~24 BPP
1024×1024 像素：约 100 万个
接龙次数：约 100 万次

自回归生成慢，多用 Diffusion 或 NAR

语音

基本单位：Sample

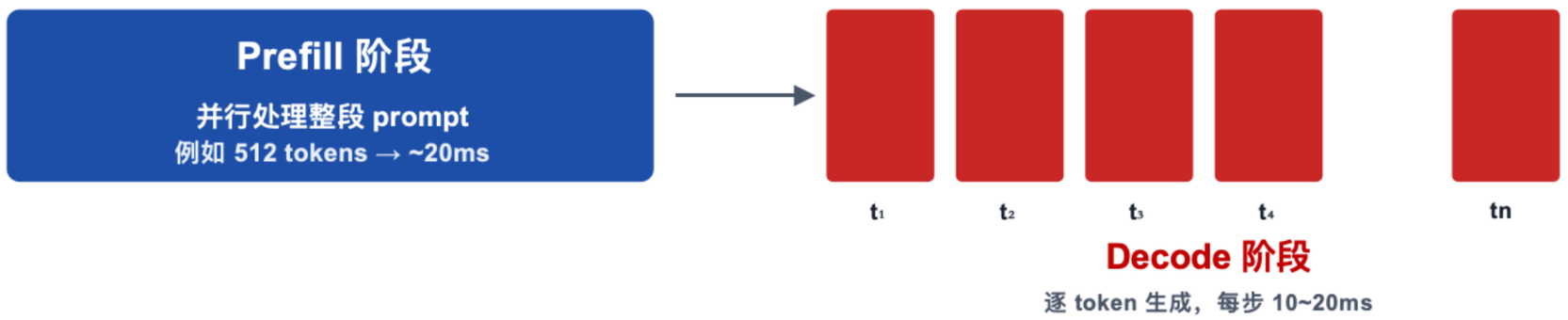
采样率：22K Hz 常见
1 分钟语音：约 130 万个采样点
接龙次数：约 130 万次

WaveNet 等使用自回归；规模最大

洞察：语言模型的"接龙"次数最少（数百次 vs 图像 100 万次 vs 语音 130 万次），这意味着它是工程优化最易出效果的模态。今天讲的所有技术，文字模态最先受益。

Prefill vs Decode：两个完全不同的瓶颈

生成过程的两阶段拆解 · Compute-bound vs Memory-bound



维度	Prefill	Decode
计算类型	大矩阵乘法（并行）	小矩阵乘法（串行）
瓶颈类型	Compute-bound（算力受限）	Memory-bound（带宽受限）
GPU 利用率	高（接近 100%）	低（10~30%，需大量优化）
主要优化	算子融合、Flash Attention	KV-Cache、量化、批处理
耗时占比	短（毫秒级）	长（占总延迟 90%+）

关键洞察：Prefill 决定首字延迟（TTFT），Decode 决定逐字延迟（TBT）。两者优化策略完全不同，必须分别处理。

为什么我们需要加快生成速度?

用数字告诉你：每秒钟都在与延迟博弈

7B 模型 · 单卡 A100 · 4K tokens

20+ s

纯 decode 耗时，已经超出人类对话舒适区

对比：ChatGPT 类产品
50~100 tokens/s，首字延迟 < 1s

175B 模型 · 单卡 A100 · 4K tokens

60+ s

单卡完全不可用，必须分布式推理

工程代价：张量并行 + 流水线并行 + KV-Cache + 量化 + 连续批处理

实时性要求（人类对话节奏）

< 5 tok/s 卡顿

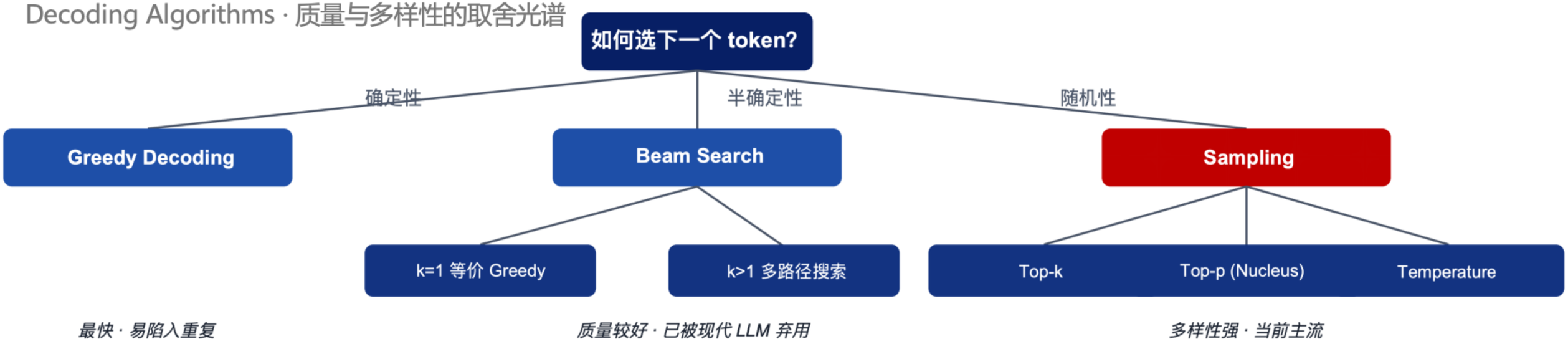
10 tok/s 可用

50+ tok/s 流畅（业界目标）

结论：业界目标 50+ tok/s；7B 单卡 4K tokens 需 20+ 秒，175B 单卡 4K tokens 需 60+ 秒。我们讲的每一项技术，都是在拉近这个差距。

Decoding 算法全景：从 Greedy 到 Sampling

Decoding Algorithms · 质量与多样性的取舍光谱



Greedy

选 argmax，单步 O(1)。
速度最快，但易陷入循环。

适合：代码、数学等需要唯一正确答案的场景。

Beam Search

保留 k 条路径，计算量 ×k。
质量稍好但代价昂贵。

现状：现代 LLM 基本不用。

Sampling

直接从分布采样。
多样性强，配合 Top-k/Top-p/Temperature。

现状：当前主流方案。

Greedy 与 Beam Search：追求最优的代价

确定性解码 · 局部最优 vs 全局搜索

Greedy Decoding

每步选 argmax p(x_t | x<t)

优点

- 速度快，单步 O(V) 计算
- 实现简单，无需额外参数
- 可复现性强

致命缺陷：神经文本退化

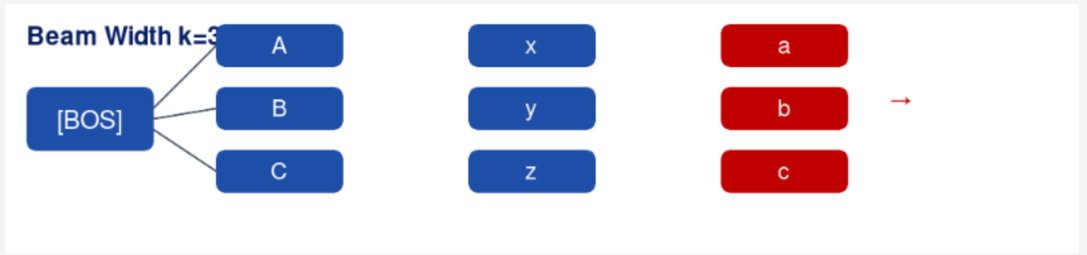
"我不知道。我不知道。我不知道……"—— Holtzman et al., 2019

最大概率序列反而最容易重复——违反"概率最高=最优"的直觉。

适用：代码生成、数学推理、低温 τ≈0

Beam Search

每步保留 top-k 候选，向下展开



优势与代价

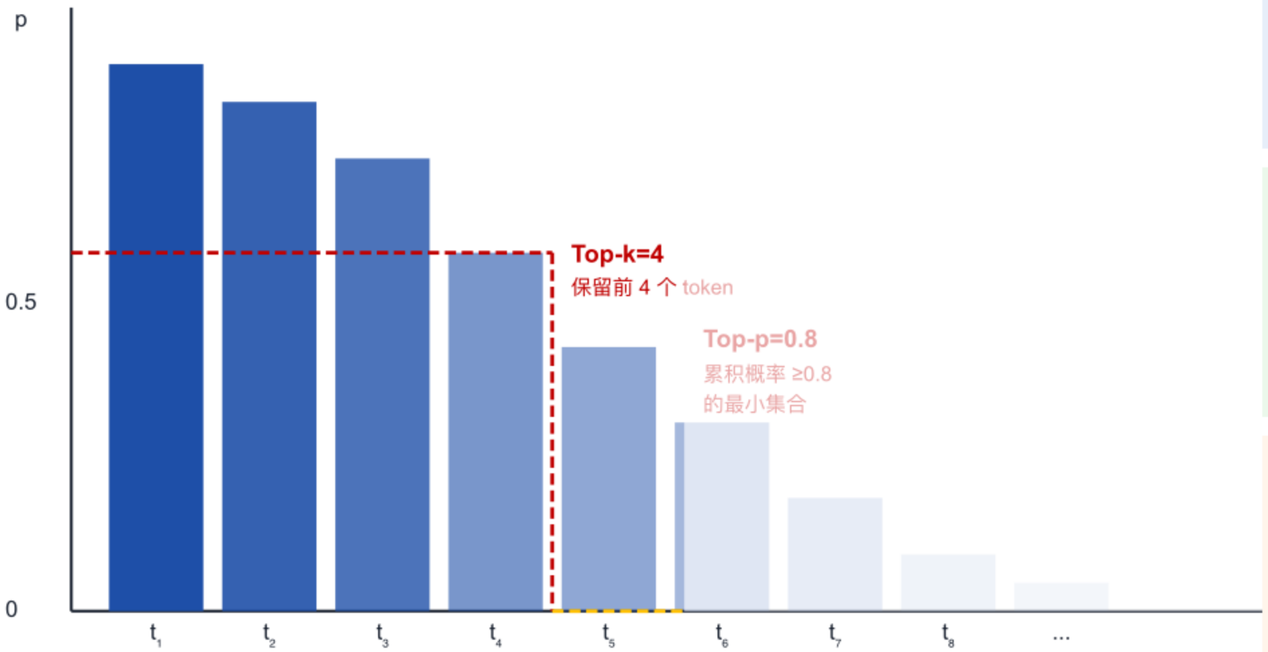
优势：近似全局最优，对机器翻译、摘要等任务有效
代价：计算量 ×k，beam width k=5 时延 ×5

现状：现代 LLM 基本不用 Beam Search。RLHF 校准后的 Greedy + 低温度已能产生高质量文本，Beam 的额外计算得不偿失。

Top-k、Top-p 与 Temperature：控制多样性的三驾马车

随机性解码 · 自适应截断与概率再分配

概率分布上的三种截断



横轴：token 序号（按概率从大到小排序）

Top-k Sampling · Fan et al., 2018

只从概率最高的 k 个 token 中采样。问题：固定 k 无法适配不同分布形态。

Top-p (Nucleus) Sampling · Holtzman et al., 2019

按累积概率截断。分布尖锐时 V' 小，平坦时 V' 大。优势：自适应，更鲁棒。

Temperature · p ∝ exp(z/T)

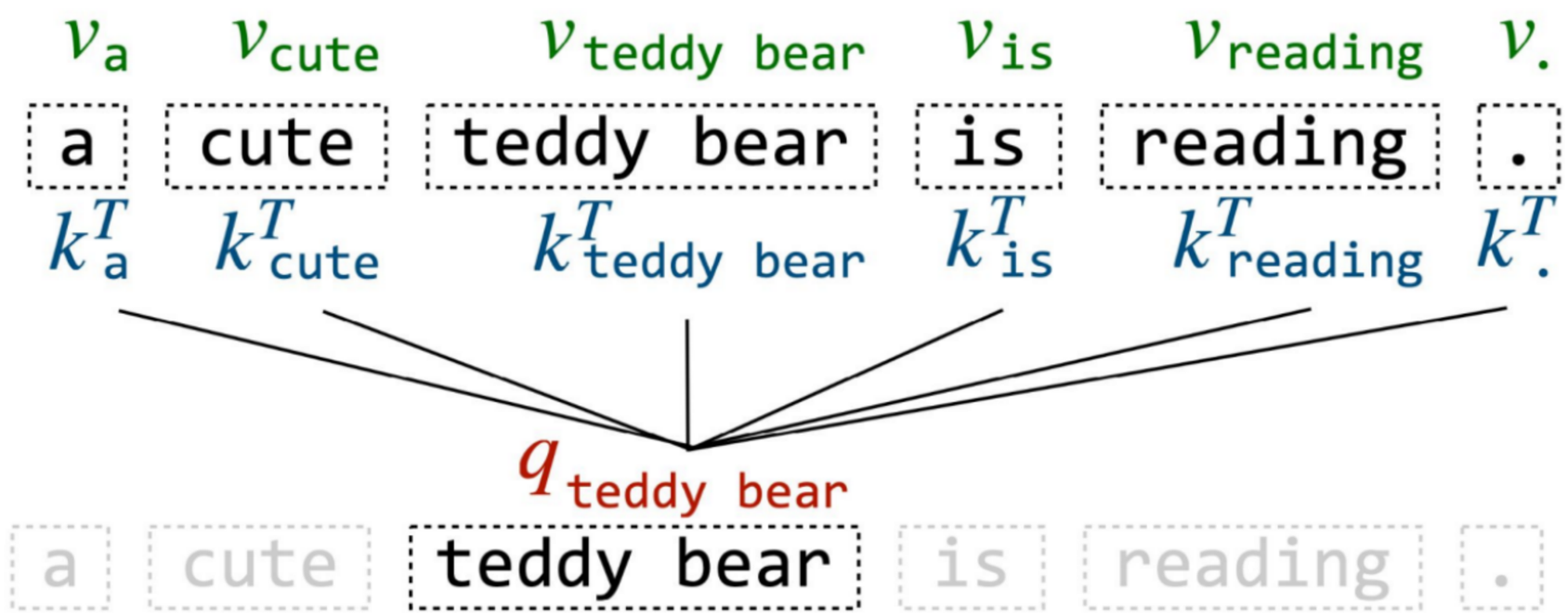
缩放 logits。T→0 接近 Greedy，T→∞ 均匀分布。它不是算法，是预处理。

新趋势：Best-of-N

生成 N 个候选，用奖励模型（或 LLM-as-judge）挑选最佳。在 reasoning 任务上效果显著，能让小模型与更大模型竞争。代价是计算量 ×N，通常 N=4~16。

模型架构

- Transformer模型：
  - 可将传统的Seq2Seq中的Encoder和Decoder中的RNN模块全部用Attention来替换
  - 一次性处理完整个序列:

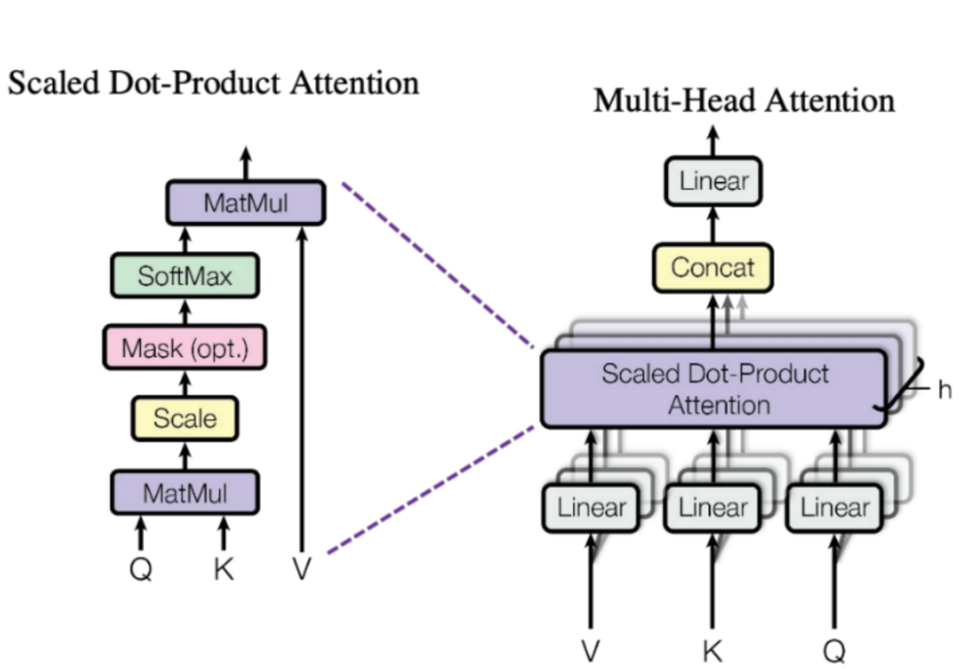


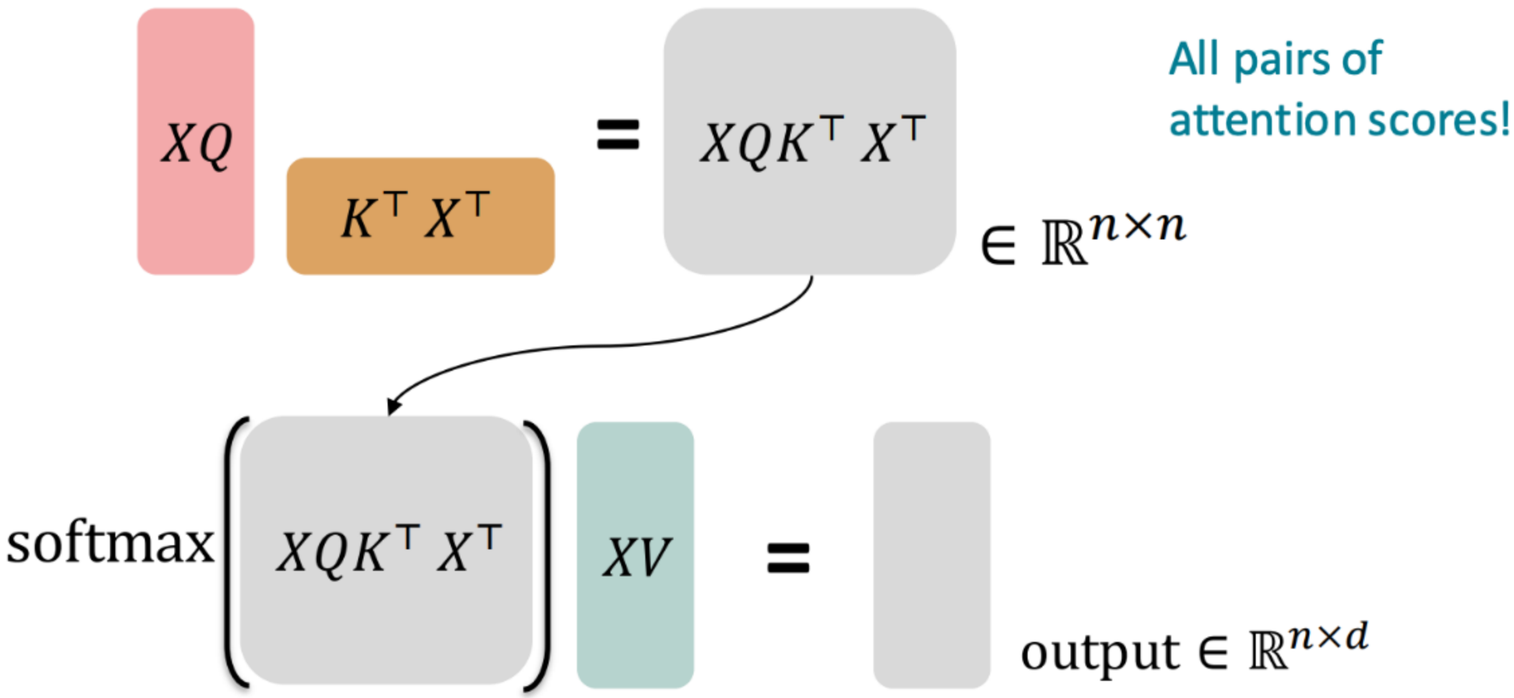
$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

来源：Attention is all you need. NIPS 2017

模型架构

- Transformer模型：
  - 可将传统的Seq2Seq中的Encoder和Decoder中的RNN模块全部用Attention来替换
  - 一次性处理完整个序列



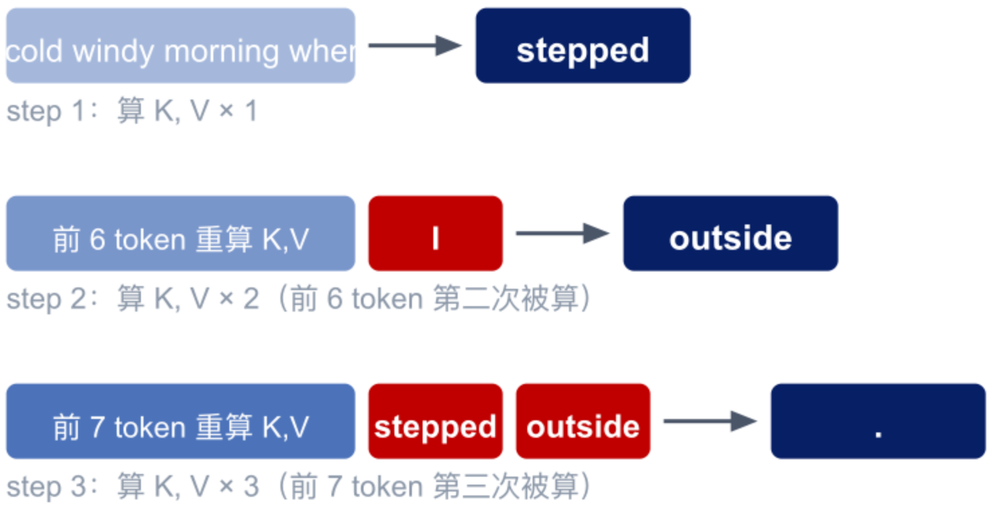


来源：Stanford CS224N

Self-Attention 的冗余：每次都重算历史

Vanilla Self-Attention · 每生成一个 token 都要重算历史的 K、V

输入："a cold windy morning when I"



⚠ 重复计算量

生成 N 个 token：

1 + 2 + 3 + ... + N

= $O(N^2)$ 次 K, V 计算

历史 token 的 K, V 反复被算
这是巨大的算力浪费

长序列下完全不可用

Self-Attention 三要素

Q (Query)：当前 token 的查询向量，用于"问问题"

K (Key)：历史 token 的"索引"，用于被查询

V (Value)：历史 token 的"内容"，被 Attention 加权求和

直觉：每生成一个新 token，
只需要做两件事：

① 计算新 token 的 Q, K, V

② 用新 Q 与历史 K, V 做 Attention

历史的 K, V 完全可以"存下来，下次直接用"——这就是 KV-Cache。

关键洞察

历史 token 的 K、V 在生成新 token 时不变——它们的值完全由历史本身决定。

一句话：存储历史 K, V 省去重复计算，把 $O(N^2)$ 压成 $O(N)$。

解决方法

把已经算好的 K、V 存下来，每步只算新 token 的 Q、K、V。这正是 KV-Cache 的核心思想——用空间换时间，把 $O(N^2)$ 的重复计算降到 O(N)。

KV-Cache 的核心思想：用空间换时间

从 O(N²) 到 O(N) · 时间-空间权衡

朴素 Decode · 每步计算量

O(N²)

每生成一个新 token 都要重算历史的 K 和 V

KV-Cache · 每步计算量

O(N)

历史的 K、V 已缓存，只算新 token 的 Q、K、V



每 token 的额外计算量：从 O(N) 降到 O(1)，但需要 O(N) 的额外显存

内存公式

单 token KV-Cache 占用 = 2 × L × d_model × precision（2 = K + V）

整条序列 = 2 × L × d_model × precision × N（N = 序列长度）

内存账本：KV-Cache 到底占多少空间?

LLaMA2-70B 真实案例 · 显存压力分析

LLaMA2-70B 参数

L (层数) = 80
d_model (hidden dim) = 8192
precision (FP16) = 2 字节
N (序列长度) = 1024 tokens
batch = 32

单条 1024 tokens KV-Cache

≈ 2.6 GB

70B 模型权重 ≈ 140 GB（FP16）

32 条 batch 的 KV-Cache ≈ 84 GB，已接近权重规模

LLaMA2-70B KV-Cache 显存热力图（GB）

	batch=1	batch=8	batch=32	batch=128
N=512	1.3 GB	10.5 GB	42 GB	168 GB
N=1024	2.6 GB	21 GB	84 GB	336 GB
N=2048	5.2 GB	42 GB	168 GB	672 GB
N=4096	10.5 GB	84 GB	336 GB	1344 GB
N=8192	21 GB	168 GB	672 GB	2.6 TB

颜色含义：< 2 GB 可接受 | 2~10 GB | 10~50 GB | > 50 GB 危险

关键洞察

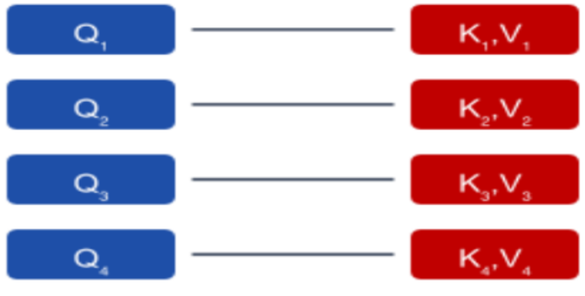
长序列 + 大 batch 是 KV-Cache 的双重暴击。生产环境必须依赖六类优化技术（GQA/MLA、量化、剪枝、Sliding Window、Paged Attention、Speculative Decoding）才能落地。

架构级优化：MQA、GQA 与 MLA

从多头独立到分组共享再到 latent 压缩

① MHA（标准多头）

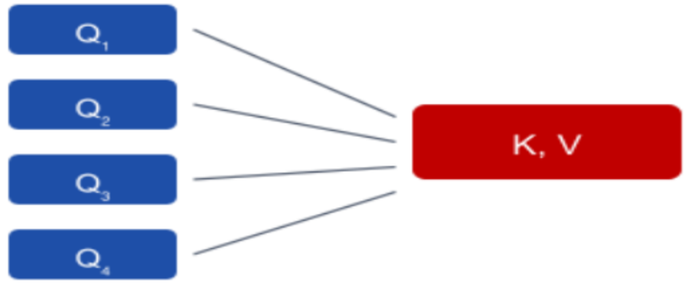
每个 Q head 独立 K、V



KV-Cache × 1

② MQA（多查询）

所有 Q head 共享一组 K、V



KV-Cache × 1/4

③ GQA（分组查询）

Q head 分组共享 K、V



KV-Cache × 1/2

④ MLA（多头潜在）

压缩到 latent 空间



只缓存 c，KV-Cache ≈ 1/16

(DeepSeek-V2/V3)

2017 MHA 2023 GQA　　2019 MQA　　2024 MLA

MQA (Shazeer, 2019)

所有 Q head 共用 1 组 KV。
KV 减到 1/h，但能力明显下降。

GQA (Ainslie, 2023)

Q head 分 g 组共享 KV。
业界标准，LLaMA-2/3、Mistral、PaLM 均采用。

MLA (DeepSeek, 2024)

压缩到 latent 向量 c。
KV-Cache 减到 GQA 的 1/4，效果无损。

演进路线：MHA → MQA → GQA → MLA。核心：压缩 KV-Cache 但保留模型能力。

内存级优化：量化与剪枝

降低精度与丢弃不重要 token 的权衡

量化：FP32 → FP16 → INT8 → INT4

FP16 — 100% 精度，无损
INT8 — 99% 精度，几乎无损
INT4 — 95~98% 精度，有损
INT2 — 极小部署，能力崩塌

LLaMA2-70B 单 token KV-Cache：
FP16: 2.6 MB · INT8: 1.3 MB · INT4: 0.65 MB

工程取舍：混合精度——大部分 KV 用 INT4，关键 token（如开头几个）用 FP16。简单粗暴但效果显著。适用：所有场景，特别是显存紧张的批量推理。

Pruning / Sparsity：保留重要 token

H2O (Zhang et al., 2023)

设定 budget，动态淘汰低 Attention 分数 token。发现：少数 token 是"heavy hitter"，反复被关注。保留 92% 能力，节省 80% 显存。

Scissorhands (2023)

更激进：观察到 token 一旦被关注一次后大概率不再被关注。基于"持续性预测"提前丢弃 KV。

核心假设：Attention 分数能反映 token 重要性。大多数场景成立，但在分布外场景（罕见代码、长尾对话）可能失效。常与量化组合使用。

注意力结构：Sliding Window 与 StreamingLLM

限制 Attention 范围 · 局部性假设与 Attention Sink

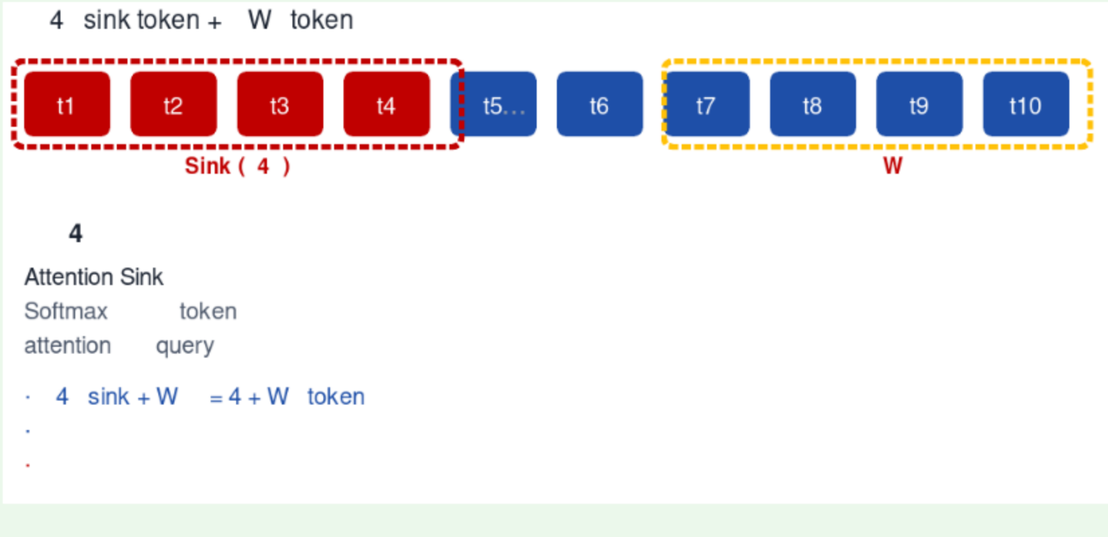
Sliding Window Attention

Mistral 7B · GPT-OSS · 窗口大小 W



收益：KV-Cache 占用降到常数，与总序列长度无关；代价：丢失长程信息。

StreamingLLM · Xiao et al., ICLR 2024



收益：理论上支持无限长序列；代价：必须保留开头几个 sink token。

共同思想

不是所有 token 都需要被 Attention。找到"重要子集"（局部窗口 + sink），就能以常数级 KV-Cache 处理任意长度序列。这是长上下文推理的基石。

硬件级优化：Flash Attention 与 Paged Attention

IO-Aware 与分页管理 · 现代推理引擎的基石

⚡ Flash Attention (Dao et al., NeurIPS 2022)

SRAM vs HBM：仓库 vs 工作台

传统方法：



Flash Attention：



小块搬运，SRAM 内完成计算

效果：

· 速度提升 2~4 倍

· 显存从 O(N²) 降到 O(N)

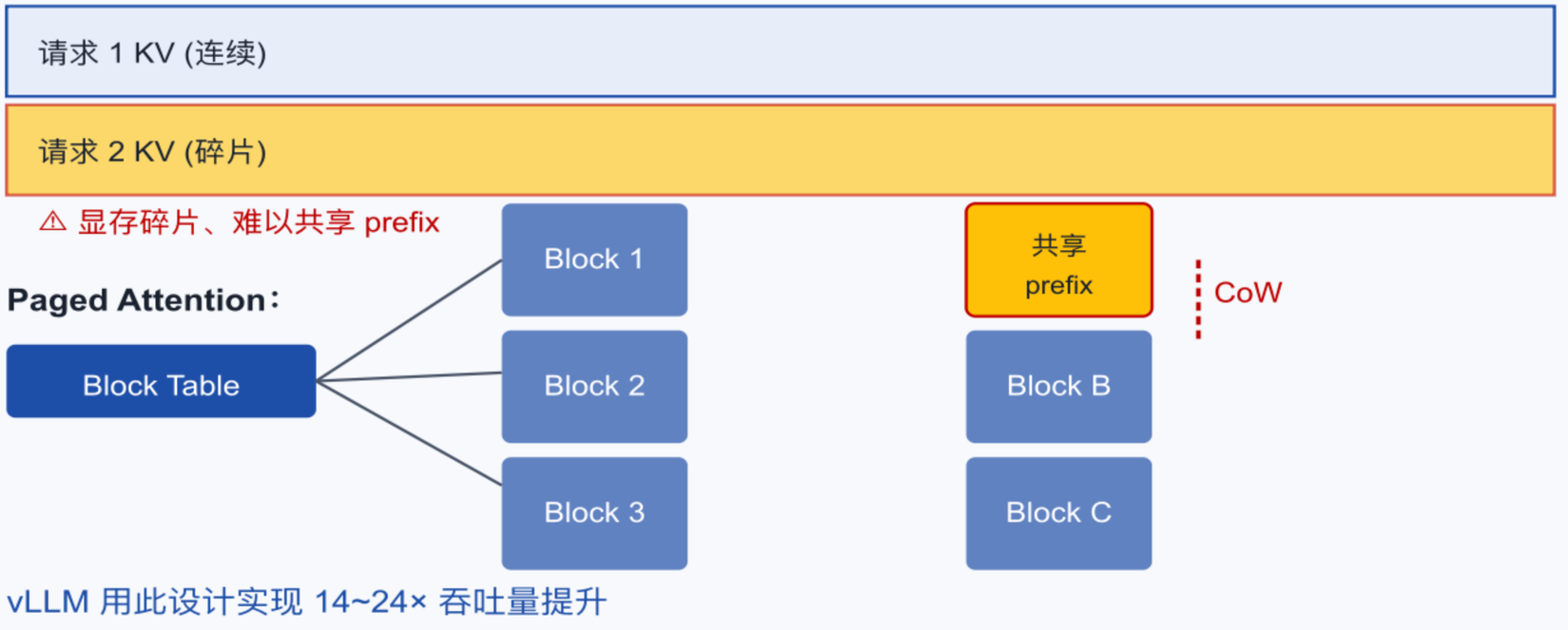
数学等价，仅重新组织计算次序；主要用于 Prefill 阶段

硬件级优化：Flash Attention 与 Paged Attention

IO-Aware 与分页管理 · 现代推理引擎的基石

Paged Attention (Kwon et al., SOSP 2023)

传统 vs Paged：连续分配 vs 分页管理



vLLM 用此设计实现 14~24× 吞吐量提升

灵感来自操作系统虚拟内存：分页 + 页表 + Copy-on-Write

共同思想

把软件问题转化为硬件/系统问题——Flash Attention 优化内存层级，Paged Attention 优化显存碎片与共享。两者已成 vLLM、SGLang 等推理引擎的默认配置。

核心动机：每个 Token 的难度并不相同

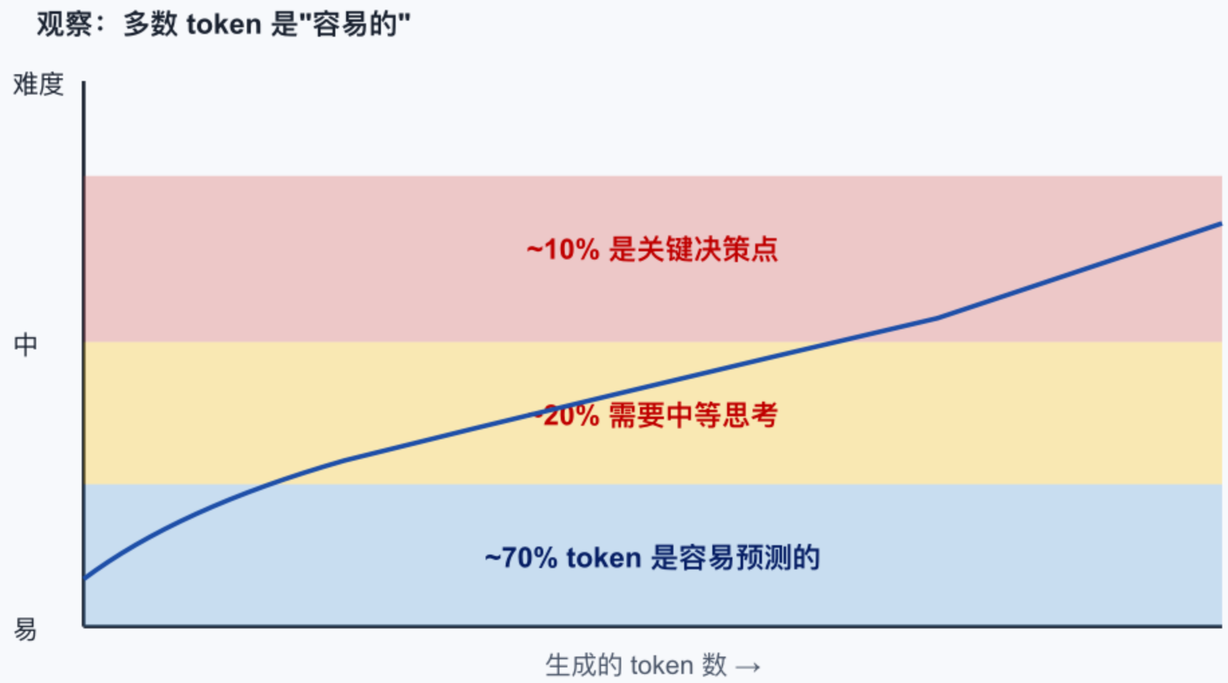
为什么我们需要小模型来"预测"大模型的输出?

2~3×

加速比 · 输出完全一致

草稿模型：T5-small (60M)
目标模型：T5-XXL (11B)
输出与原方法 bit-for-bit 等价

生成序列中 token 的难度分布



关键洞察

如果小模型能猜对大部分容易的 token，大模型只需要"验证"它们——这就是 2~3× 加速的来源。Speculative Decoding 把"逐 token 串行"变成"草稿 + 验证并行"，一次 forward 出多个 token。

工作机制：小模型猜测 + 大模型验证

三步流程：Draft → Verify → Accept/Reject

输入上文

① Draft（草稿） — 小模型 1B

小模型快速生成 K=4 个候选 token

dogs | love | chasing | after | ...

② Verify（验证） — 大模型 100B

一次性 forward，并行计算 4 个位置的 q 分布

q_1=0.9 | q_2=0.8 | q_3=0.3 | q_4=0.8

③ Accept / Reject（接受 / 拒绝）

按 acceptance sampling 逐个比较

✓ dogs | ✓ love | ✗ chasing (从这重新生成)

最终接受： dogs | love → 然后从 "chasing" 之后重新生成

① Draft · 草稿

小模型快速生成 K 个候选 token。
典型 K=4~8。

小模型参数少，计算极快，1~2ms 可完成。

② Verify · 验证

K 个候选拼成序列，一次 forward。
并行算 K 个位置的 q 分布。

关键：一次大模型前向，验证所有候选。

③ Accept/Reject

逐 token 比较 p 与 q 的概率。
接受则继续；拒绝则重采样。

接受率 α 决定最终加速比。

加速根源：一次大模型 forward = 多个 token 验证，总计算量几乎不变 → 接受率 α 决定加速比。

Acceptance Sampling：概率论的精妙之处

为什么 Speculative Decoding 输出完全等价?

核心公式

$$\text{acceptance probability} = \min\left(1,\ \frac{q(\tilde{x})}{p(\tilde{x})}\right)$$

具体例子：上下文"我想养只"

大模型 q 分布： 猫 0.6 · 狗 0.3 · 鸟 0.1
小模型 p 分布（预测"狗"）： 猫 0.4 · 狗 0.5 · 鸟 0.1

接受"狗"的概率 = min(1, q(狗)/p(狗)) = min(1, 0.3/0.5) = **0.6**

如果拒绝 → 从 q 中重采样，得到"猫/狗/鸟"，概率 0.6/0.3/0.1。

数学保证

尽管中间用到了小模型分布 p，整个过程最终严格从大模型分布 q 采样。

这意味着输出分布完全等价于直接用大模型生成。

关键洞察

Speculative Decoding 不损失任何质量，是"无损加速"。这一点和量化、剪枝等有损方法有本质区别。证明思路：通过重要性采样（Importance Sampling）推导，最终分布 = q(x)。完整证明见 Chen et al., 2023, Appendix I。

Acceptance Sampling：概率论的精妙之处

- Now, we can compare the probability of each token assigned by draft model M_p and target model M_q

		y_1	y_2	y_3	y_4	y_5
		dogs	love	chasing	after	cars
Draft model (1B)	p_i	0.8	0.7	0.9	0.8	0.7
Target model (100B)	q_i	0.9	0.8	0.8	0.3	0.8

- Starting from y_1, decide whether to accept the tokens generated by the draft model.
- Case 1: $q_i \geq p_i$
  The target model (100B) likes this token, even more than the draft model.
  => Accept this token!

Generation after step 1:
dogs

Acceptance Sampling：概率论的精妙之处

- Now, we can compare the probability of each token assigned by draft model M_p and target model M_q

		y_1	y_2	y_3	y_4	y_5
		dogs	love	chasing	after	cars
Draft model (1B)	p_i	0.8	0.7	0.9	0.8	0.7
Target model (100B)	q_i	0.9	0.8	0.8	0.3	0.8

- Starting from y_1, decide whether to accept the tokens generated by the draft model.
- Case 2: $q_i < p_i$ (accept)
  Target model doesn't like this token as much as the draft model...
  => Accept it with the probability $\frac{q_i}{p_i}$

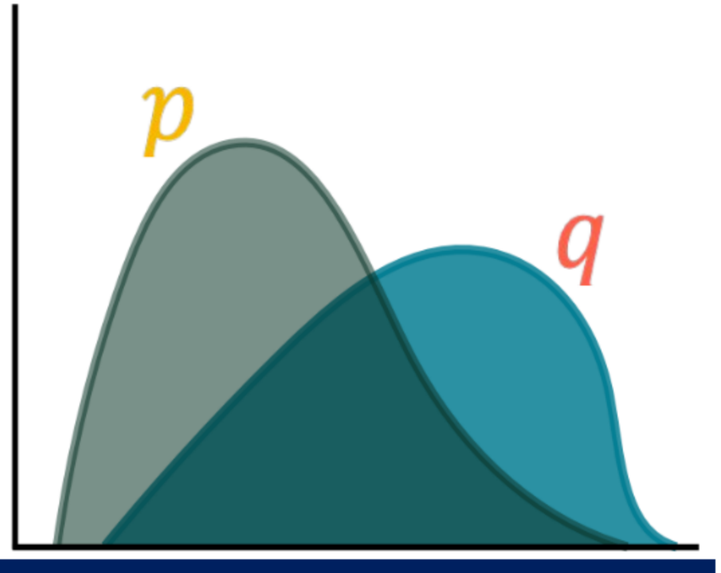
Generation after step 3:
dogs love chasing
(assuming we accepted chasing w/ prob 0.8/09)

Acceptance Sampling：概率论的精妙之处

- Now, we can compare the probability of each token assigned by draft model M_p and target model M_q

		y_1	y_2	y_3	y_4	y_5
		dogs	love	chasing	after	cars
Draft model (1B)	p_i	0.8	0.7	0.9	0.8	0.7
Target model (100B)	q_i	0.9	0.8	0.8	0.3	0.8

- Starting from y_1, decide whether to accept the tokens generated by the draft model.
- Case 3: $q_i < p_i$ (reject)
  If $q_i <<< p_i$, we likely would have rejected it.
  In this case, we sample a new token from target model
  => Specifically, we sample from $(q_i - p_i)_+$



工程前沿：Medusa、EAGLE 与 Suffix Decoding

Speculative Decoding 的三条演化路线

M · Medusa (2023)

- 在大模型上加多个预测头
- 每个头预测未来某位置
- 无需单独草稿模型

共享主干，省一份模型加载。

E · EAGLE-3 (2025/2026)

- 草稿模型偷看目标模型内部特征
- EAGLE-2: 动态草稿树
- EAGLE-3: 多层特征融合

当前最强通用加速方案。

S · Suffix Decoding

- 无模型：Suffix Tree 匹配
- ~20μs/token（超快）
- 适合重复性强的任务

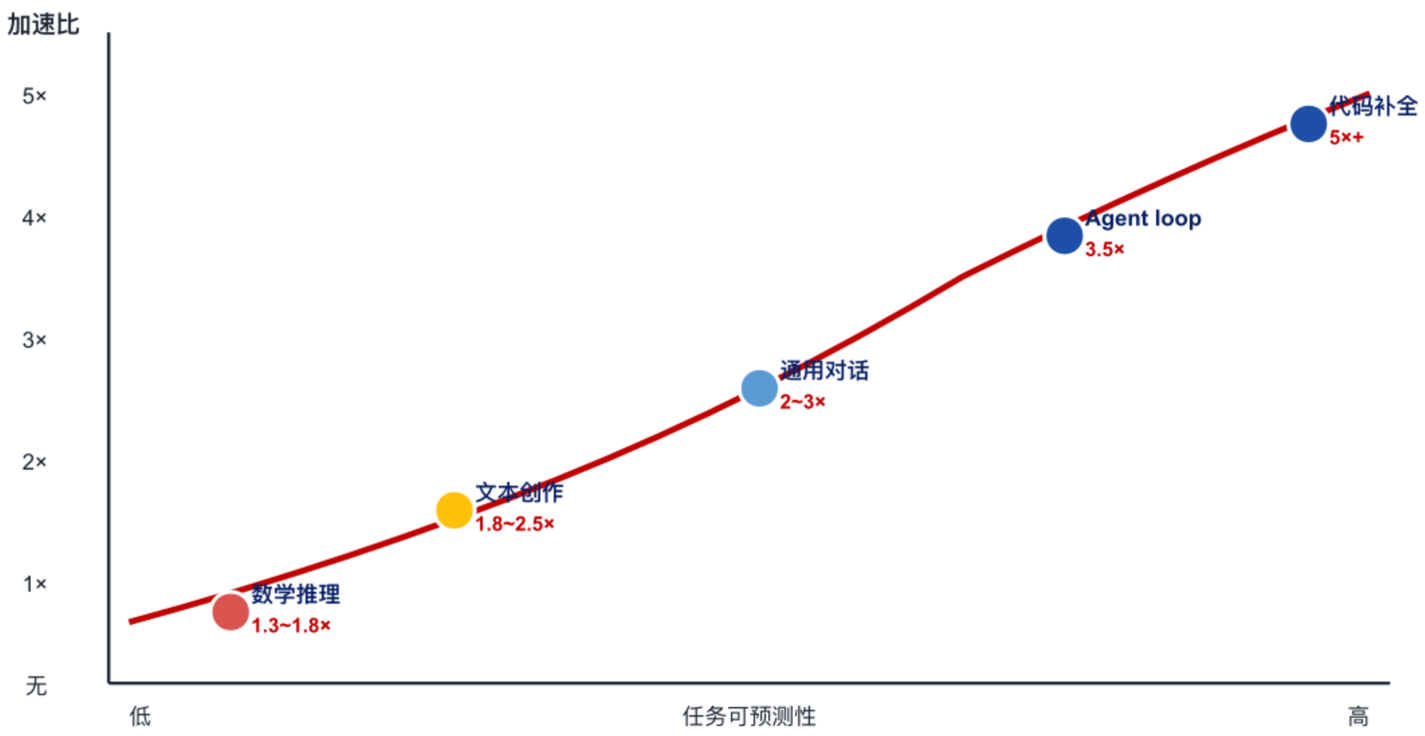
Oliaro et al., 2025

算法	机制	最佳场景	训练需求
Draft–Target	小模型+大模型	通用场景	需同 tokenizer
Medusa	附加预测头	通用 / 多头融合	需训练预测头
EAGLE–3	特征级草稿	开放性任务 / 通用	需训练草稿头
Suffix Decoding	Suffix Tree 匹配	代码 / Agent / RAG / SQL	零训练

选择策略：任务重复性强 → Suffix Decoding（零训练、超快）；通用任务 → EAGLE-3（最强通用加速）。

实测效果与适用场景

加速比 vs 任务可预测性 · 何时使用，何时不用



曲线规律：任务越模板化、重复性越高，加速效果越好

T5-small + T5-XXL

Chen et al., 2023 · 60M 草稿 + 11B 目标 · 加速 2~3×，输出等价

Medusa / EAGLE-3

7B/13B 模型常用方案 · 加速 2~4×，需训练辅助头

Suffix Decoding

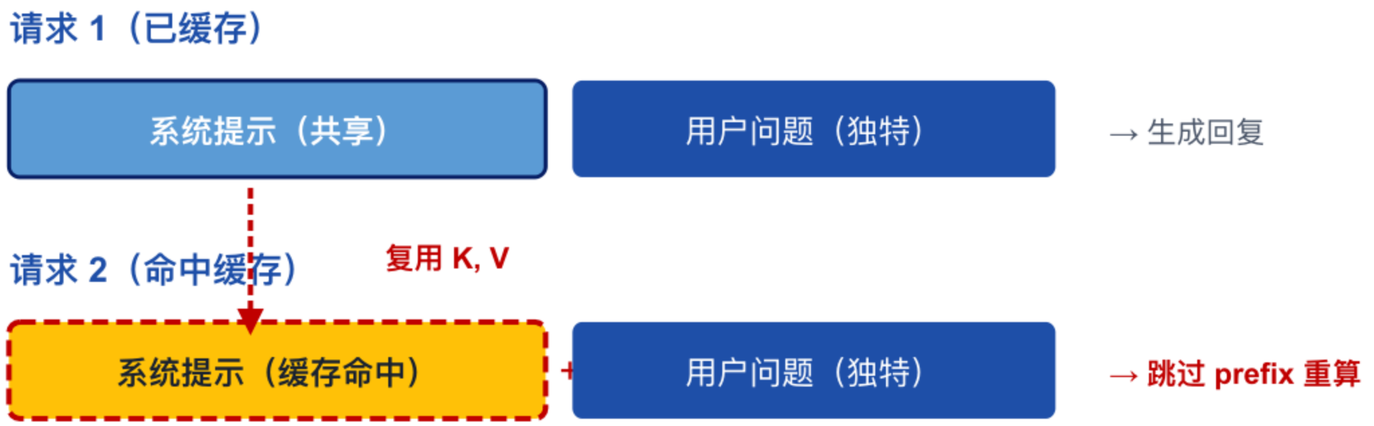
代码补全 / Agent loop · 加速 5×+，零训练

选择建议

草稿模型大小 ≈ 目标的 1/15~1/20，相同数据训练效果最佳；小 batch 收益最大。任务越模板化，接受率越高，加速比越大——代码 > Agent 工具调用 > 结构化输出 > 开放对话 > 创意写作。

系统级优化：Prompt Caching

重复前缀共享 KV-Cache · 系统层面的"免费午餐"



应用场景：AI Agent 调用

每次调用都附上 system prompt + 工具定义 + 长期记忆

- System Prompt（稳定不动）
- 工具定义（稳定不动）
- 长期记忆（稳定不动）
- 用户当前问题（变化）
- 上下文消息（变化）
- 工具调用结果（变化）

设计原则：稳定不动的放前面，可能变动的放后面

问题

多轮对话：system prompt 每次都重算；RAG：文档前缀反复出现；Agent：system + tools 定义长且固定。

方案

识别可共享前缀，KV-Cache 持久化在显存中。新请求只计算新增部分。

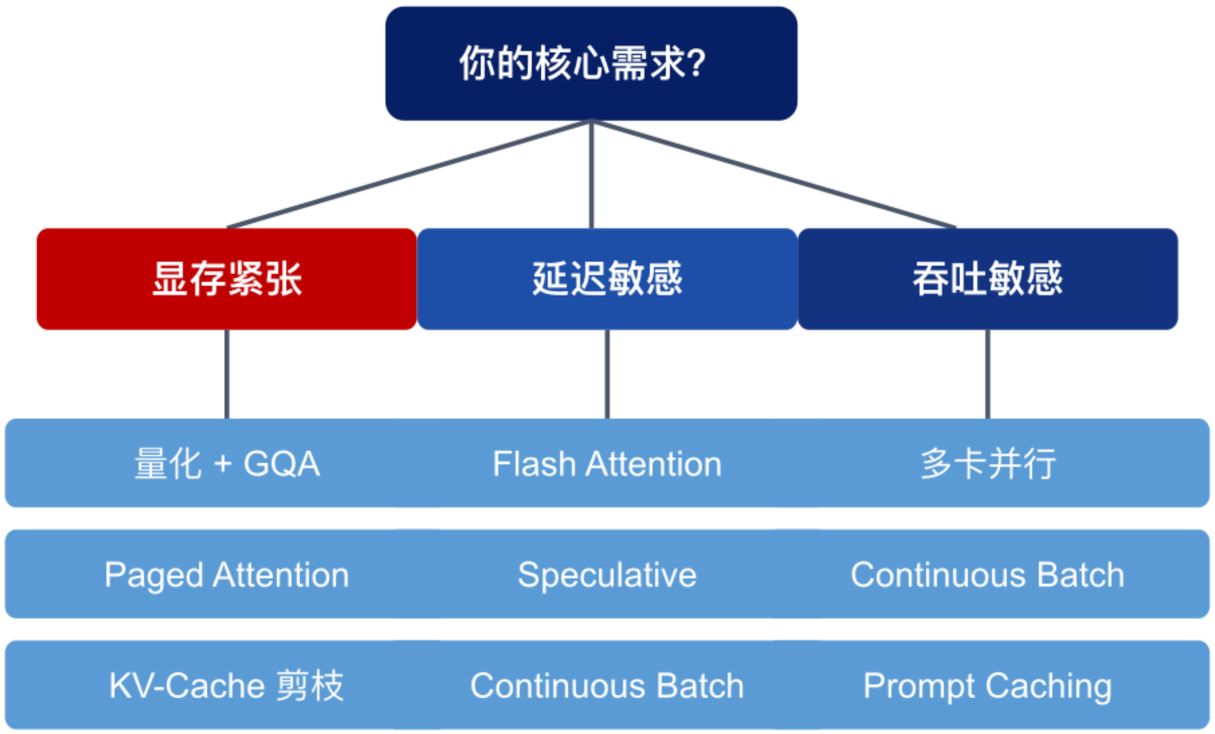
效果

Prefill 延迟降低 80~90%。Anthropic、OpenAI 已默认开启。企业 RAG 场景平均节省 50%+ 首 token 延迟。

注意：前缀必须严格一致（token 级精确匹配），任何空格/标点差异都会导致缓存未命中。

加速方案全景对照表

六类技术 · 无损 vs 有损 · 何时用哪个?



⚠ **单一技术收益有限**
工程上是组合拳

技术	有损?	倍数
GQA/MLA	无损	~1.3×
量化	微损	~2×
剪枝	有损	~1.5×
Sliding	有损	~2×
Flash	无损	~2×
Paged	无损	~3×
Spec	无损	~3×
Prompt Cache	无损	~5×

组合使用原则

生产环境 = GQA/MLA + 量化 + Flash + Paged + Prompt Cache + Speculative Decoding。单项 2~3×，组合可达 10~30×。

加速部分总结

加快语言模型生成速度 · 本讲回顾



① **自回归是根本瓶颈**：Decode 阶段每生成一个 token 必须串行跑一次 forward，batch 利用率低。Prefill/Decode 两阶段的矛盾是所有优化的出发点。
② **KV-Cache 是内存瓶颈**：70B 模型单序列 2.6GB，长上下文 + 大 batch 直接爆显存。GQA/MLA、量化、剪枝、Sliding Window 都是为了压 KV。
③ **Flash/Paged 是系统基石**：IO 感知计算 + 分页显存管理，让 GPU 算力利用率从 ~10% 提升到 ~50%。vLLM/SGLang 的核心。
④ **Speculative Decoding 是无损飞跃**：小模型草稿 + 大模型验证，Acceptance Sampling 保证输出严格等价。EAGLE-3 已实现 3~4× 无损加速。
⑤ **工程是组合拳**：单项 2~3×，组合 10~30×。生产环境 = GQA/MLA + 量化 + Flash + Paged + Prompt Cache + Spec Decoding。